



Completeness of Iris-Based Program Logics

JOHANNES HOSTERT*, ETH Zurich, Switzerland
ZICHEN ZHANG*, New York University, USA
PUMING LIU, NYU Shanghai, China
SIMON ODDERSHEDE GREGERSEN, CISPA Helmholtz Center for Information Security, Germany
RALF JUNG, ETH Zurich, Switzerland
JOSEPH TASSAROTTI†, New York University, USA

Traditionally, proof systems such as program logics come with two core theorems: *soundness* and *completeness*. The role of soundness is obvious: we want to be sure that arguments carried out inside the logic actually lead to correct conclusions. Completeness complements that by ensuring that the logic does not limit expressivity: in principle, any correct result can be obtained within the confines of the logic. This result typically has to be stated *relative* to the completeness of the assertion logic that is used to reason about pre- and postconditions.

Over the past decade, the Iris framework has emerged as a widely used foundation for building separation logics. While Iris-based logics typically come with a soundness proof, none of them have had a proof of completeness. In this paper, we present the first approach for establishing completeness of Iris-based program logics, and we show the generality of our methodology by applying it to a range of different logics described in prior work, including partial and total concurrent separation logics for a higher-order ML-like language, two quantitative logics (for bounding execution time and probabilistic errors), and a relational logic for proving refinement. All our results have been mechanized in the Rocq prover.

CCS Concepts: • **Theory of computation → Separation logic**; **Logic and verification**; **Program verification**; *Probabilistic computation.*

Additional Key Words and Phrases: Completeness, Data Races, Time Credits, Error Credits, Relational Logic

ACM Reference Format:
Johannes Hostert, Zichen Zhang, Puming Liu, Simon Oddershede Gregersen, Ralf Jung, and Joseph Tassarotti. 2026. Completeness of Iris-Based Program Logics. *Proc. ACM Program. Lang.* 10, ICFP, Article 284 (August 2026), 36 pages. https://doi.org/10.1145/3828682

1 Introduction

Iris [36, 38, 41] is a higher-order separation logic framework that has been used for numerous program logics and verification projects. It provides reusable building blocks for constructing different variants of separation logic, with applications ranging from verification tools for C [50, 61], Rust [16, 51], OCaml [2, 24, 62], and Go [9], to reasoning about crash recovery [8], distributed execution [44, 63, 67], weak memory [13, 39, 53], resource consumption [56, 60], probabilistic

*Joint first authors.
†Also affiliated with Amazon Web Services. This paper does not reflect the views of Amazon Web Services.

Authors' Contact Information: Johannes Hostert, ETH Zurich, Department of Computer Science, Zurich, Switzerland, johannes.hostert@inf.ethz.ch; Zichen Zhang, New York University, New York, USA, zichenzhang@nyu.edu; Puming Liu, NYU Shanghai, Shanghai, China, pl2559@nyu.edu; Simon Oddershede Gregersen, CISPA Helmholtz Center for Information Security, Saarbrücken, Germany, gregersen@cispa.de; Ralf Jung, ETH Zurich, Department of Computer Science, Zurich, Switzerland, ralf.jung@inf.ethz.ch; Joseph Tassarotti, New York University, New York, USA, jt4767@nyu.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2026 Copyright held by the owner/author(s).
ACM 2475-1421/2026/8-ART284
https://doi.org/10.1145/3828682

programs [1, 21, 22, 25, 26, 47], and semantic models of challenging type system features [15, 18–20, 23, 28, 35, 55, 68, 70]. All of these projects come with foundational soundness proofs, mechanized in the Rocq prover, that build on or adapt the core soundness proofs provided by the framework itself. These soundness results show that proofs of specifications about programs in the logic actually imply the intended properties about programs' executions.

Although the many applications of Iris-based logics provide some empirical evidence that these logics are highly expressive, there are no formal results that characterize the expressivity of these logics. In particular, it is unknown whether these logics are *complete*, that is, whether any program whose execution satisfies certain properties can be provably shown to do so by applying the proof rules offered by these logics. Resolving this question is important because it tells us whether a logic is "missing" any rules, or whether the rules in principle suffice to verify any program.

For classical Hoare logic, completeness was long ago established by Cook [12]. This result is "relative" to the completeness of the assertion logic that is used for the rule of consequence. Subsequent works developed similar completeness results for logics for concurrency, such as the method of Owicki and Gries [59], shown complete by Owicki [58], and rely-guarantee reasoning [71]. Most recently, de Boer and Hiep [14] have shown how to adapt this style of approach to concurrent separation logic. However, these works consider languages and logics that are quite different from Iris. In prior work on completeness of concurrent program logics, the setting is a first-order language, and the only source of looping behavior is a `while`-like construct. The logics are also first-order with a strict stratification between assertion logic and specification logic. In contrast, Iris is often used to reason about higher-order languages that can encode Landin's knot [45], and the logic makes no distinction between assertions and specifications by supporting Hoare triples in pre- and postconditions, a crucial feature for reasoning about higher-order programs. As a result, it is not a priori clear how to adapt existing completeness results to Iris-based logics.

This paper introduces a general recipe for proving completeness of Iris-based program logics. In doing so, we follow the usual approach of Iris: we provide reusable building blocks that can often be directly applied, and that can be further customized when needed. Concretely, we establish a core reusable lemma for obtaining completeness proofs for instances of Iris's language-agnostic default program logic. This nicely complements the existing language-agnostic soundness proof for said logic. We also demonstrate that the proof pattern behind this lemma generalizes to other Iris-based program logics that are not a direct instance of this framework.

Using our methodology, we prove, for the first time, that a number of Iris-based logics are complete (or can be made so by adding one or two missing proof rules). Our case studies include partial and total higher-order concurrent separation logics, two quantitative logics (for bounding execution time and probabilistic errors), and a relational logic for proving refinement. As expected, these proofs make use of the core reasoning principles that Iris provides. In particular, Iris's general notion of custom ghost state allows us to characterize the reachable states of concurrent programs in a simple, language-agnostic way. This can be viewed as a generalization of the very language-specific introduction of auxiliary variables used in early work on completeness of concurrency logics. Meanwhile, *Löb induction* [48], the fundamental primitive for recursive reasoning in step-indexed logics such as Iris, is powerful enough to let us show completeness of logics for languages with higher-order state. We demonstrate this with a series of case studies. All our results have been mechanized in the Rocq prover [66] using the Iris Proof Mode [40, 42].

We begin this paper in §2 by building an Iris-style program logic for a sequential language with higher-order state and proving it complete. In §3, we explain how Iris is used to reason about concurrent programs, and we extend the completeness proof to that setting. We continue with a series of completeness case studies: the logic for Iris's default language HeapLang (§4) and its total variant (§5); λ_{Rust}, which has a non-standard memory model to rule out data races (§6); a logic

$$v, w \in \mathit{Val} ::= z \mid \ell \mid \lambda x.\, e \mid () \qquad (z \in \mathbb{Z}, \ell \in \mathit{Loc})$$
$$e \in \mathit{Expr} ::= v \mid x \mid \lambda x.\, e \mid e_1\ e_2 \mid e_1 \circledcirc_2 e_2 \mid \mathtt{if}\ e\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 \mid \quad (\circledcirc_2 \in \{+, -, =, \cdots\})$$
$$\mathtt{ref}(e_1) \mid \mathtt{free}(e) \mid !\, e \mid e_1 \leftarrow e_2$$
$$K \in \mathit{Ctx} ::= \bullet \mid e\ K \mid K\ v \mid e \circledcirc_2 K \mid K \circledcirc_2 v \mid \mathtt{if}\ K\ \mathtt{then}\ e_1\ \mathtt{else}\ e_2 \mid$$
$$\mathtt{ref}(K) \mid \mathtt{free}(K) \mid !\, K \mid e \leftarrow K \mid K \leftarrow v$$
$$\sigma \in \mathit{State} \triangleq \mathit{Loc} \xrightarrow{\text{fin}} \mathit{Val}, \quad \rho \in \mathit{Conf} \triangleq \mathit{Expr} \times \mathit{State}$$

PureBeta
$$(\lambda x.\, e)\ v \to_{\mathtt{pure}} e[v/x]$$

PureOp
$$\frac{z_1 \circledcirc_2 z_2 = z_3}{z_1 \circledcirc_2 z_2 \to_{\mathtt{pure}} z_3}$$

$\cdots$

BasePure
$$\frac{e_1 \to_{\mathtt{pure}} e_2}{(e_1, \sigma) \to_{\mathtt{base}} (e_2, \sigma)}$$

BaseAlloc
$$\frac{\ell \notin \mathrm{dom}(\sigma)}{(\mathtt{ref}(v), \sigma) \to_{\mathtt{base}} (\ell, \sigma[\ell \leftarrow v])}$$

BaseFree
$$\frac{\ell \in \mathrm{dom}(\sigma)}{(\mathtt{free}(\ell), \sigma) \to_{\mathtt{base}} ((), \sigma \setminus \{\ell\})}$$

BaseLoad
$$\frac{\sigma(\ell) = v}{(!\, \ell, \sigma) \to_{\mathtt{base}} (v, \sigma)}$$

BaseStore
$$\frac{\ell \in \mathrm{dom}(\sigma)}{(\ell \leftarrow v, \sigma) \to_{\mathtt{base}} ((), \sigma[\ell \leftarrow v])}$$

StepCtx
$$\frac{(e_1, \sigma) \to_{\mathtt{base}} (e_2, \sigma')}{(K[e_1], \sigma) \to (K[e_2], \sigma')}$$

Fig. 1. Syntax and Semantics of SeqLang.

with time credits for reasoning about execution cost (§7); a probabilistic logic for reasoning about error bounds (§8); and finally a relational logic for proving refinement of concurrent higher-order programs (§9). We finish the technical contribution of the paper with a discussion of a general semantic condition that all complete Iris logics have to satisfy (§10). We then survey prior proofs of completeness for other program logics (§11) and conclude with a discussion about the significance of completeness (§12).

2 Warm-up: Completeness for a Sequential Language

In this section, we slowly build up to our main completeness result by first considering an Iris-style program logic for the simpler, sequential setting. Concretely, we define a language SeqLang, a call-by-value lambda calculus with integers and mutable higher-order state. The syntax and semantics can be found in Figure 1. We define a small-step operational semantics with the reduction relation $\to$ using evaluation contexts K and a *base* reduction relation $\to_{\mathtt{base}}$. The definition of evaluation contexts induces a right-to-left evaluation order.

The step rules operate on configurations ρ, which are pairs consisting of an expression e and a heap σ. The heap is represented as a finite partial map from locations to values. The reduction of the pure, deterministic fragment of the language is described by the *pure* reduction relation $\to_{\mathtt{pure}}$, which can (using BasePure) be turned into a base reduction step that leaves the heap untouched. Note that Figure 1 omits some of the pure reductions for brevity. The semantics of the load and store operations are standard. Deallocation removes the heap cell (which can then be reused by future allocations). For all of these operations, the reduction relation gets stuck if the location is not allocated. Allocation is the only non-deterministic operation in SeqLang since it picks an arbitrary fresh location $\ell \notin \mathrm{dom}(\sigma)$.

$$P, Q \in \mathit{iProp} ::= \ulcorner \varphi \urcorner \mid P \land Q \mid P \lor Q \mid \forall x.\, P \mid \exists x.\, P \mid \cdots \qquad (\varphi \text{ a meta-level proposition})$$
$$\mid P * Q \mid P \mathrel{-\!\!*} Q \mid \ell \mapsto v \mid \mathsf{wp}\ e\ \{v.\, P\} \mid \triangleright P$$

PointsToNe
$\ell_1 \mapsto v_1 * \ell_2 \mapsto v_2 \vdash \ulcorner \ell_1 \neq \ell_2 \urcorner$

WpWand
$\mathsf{wp}\ e\ \{\Phi\} * (\forall x.\, \Phi(x) \mathrel{-\!\!*} \Psi(x)) \vdash \mathsf{wp}\ e\ \{\Psi\}$

WpValue
$\Phi(v) \dashv\vdash \mathsf{wp}\ v\ \{\Phi\}$

WpBind
$\mathsf{wp}\ e\ \{v.\, \mathsf{wp}\ K[v]\ \{\Phi\}\} \dashv\vdash \mathsf{wp}\ K[e]\ \{\Phi\}$

WpPure
$$\frac{e_1 \to_{\mathsf{pure}} e_2}{\triangleright\, \mathsf{wp}\ e_2\ \{\Phi\} \vdash \mathsf{wp}\ e_1\ \{\Phi\}}$$

WpAlloc
$\triangleright \forall \ell.\, \ell \mapsto v \mathrel{-\!\!*} \Phi(\ell) \vdash \mathsf{wp}\ \mathtt{ref}(v)\ \{\Phi\}$

WpFree
$\ell \mapsto v * \triangleright \Phi(()) \vdash \mathsf{wp}\ \mathtt{free}(\ell)\ \{\Phi\}$

WpLoad
$\ell \mapsto v * \triangleright(\ell \mapsto v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\ !\ell\ \{\Phi\}$

WpStore
$\ell \mapsto v * \triangleright(\ell \mapsto w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\ \ell \leftarrow w\ \{\Phi\}$

Fig. 2. A separation logic for SeqLang.

2.1 A Sequential Separation Logic

Figure 2 defines a separation logic for SeqLang. This separation logic can be used to derive judgments of the shape $P \vdash Q$, where P and Q are separation logic assertions. Separation logic enriches propositional logic with the separating conjunction $*$ and separating implication $\mathrel{-\!\!*}$, sometimes called the "magic wand". The separating conjunction $P * Q$ expresses ownership of P and Q *separately, i.e.*, P and Q hold for disjoint parts of the heap. In particular, $P \vdash P * P$ does not always hold. Most often, Iris-based separation logics are *affine* logics, meaning they admit the weakening rule $P * Q \vdash P$. The separating implication $\mathrel{-\!\!*}$ is a version of implication that interacts with separating conjunction the same way that regular implication interacts with conjunction: $P \vdash Q \mathrel{-\!\!*} R$ iff $P * Q \vdash R$. The assertion $\ulcorner \varphi \urcorner$ *embeds* an arbitrary meta-level proposition φ (*i.e.*, a Rocq `Prop`) into the syntax of the logic. In particular, if $\varphi \Rightarrow \varphi'$ holds at the meta-level then $\ulcorner \varphi \urcorner \vdash \ulcorner \varphi' \urcorner$. The later modality $\triangleright$ may be ignored for now. We refer to Chapter 3 of Birkedal and Bizjak [5] for a detailed expository introduction to separation logic.

We give meaning to our connectives "axiomatically" using a natural deduction-style derivation system. The full rules are given in Appendix C, which are precisely the interface Iris exposes for its `uPred` model.[1] Due to the embedding of meta-level propositions $\ulcorner \varphi \urcorner$, these rules are not purely syntactic.

To turn our logic into a program logic, we first introduce the points-to connective $\ell \mapsto v$. This assertion expresses that the current heap stores the value v at location ℓ. Furthermore, this fact is *exclusive*, meaning it is not possible to own two points-to connectives for the same piece of state. This is witnessed by the rule PointsToNe.

We reason about program executions using the assertion $\mathsf{wp}\ e\ \{v.\, P\}$. Here, v serves as a binder for the return value in postcondition P. We use $\mathsf{wp}\ e\ \{\Phi\}$ as syntactic sugar for $\mathsf{wp}\ e\ \{v.\, \Phi(v)\}$. Like all separation logic assertions, this is an assertion about the current program state: Intuitively, $\mathsf{wp}\ e\ \{v.\, P\}$ holds in some program state if executing e from that state never gets stuck, and if

[1] Strictly speaking, the Iris Rocq development does not formally define a syntactic derivation system. Instead, it defines a semantic model of assertions that validates a set of *base logic rules* [36]. It then makes the model definition *opaque*, so that client proofs are carried out only using these rules as if they were a syntactic system. In our Rocq development, we thus treat these base lemmas as the abstract deduction system and formally prove that they are complete.

furthermore whenever e terminates with a value w, then $P[w/v]$ holds for the final state. This property expresses a form of partial correctness as it does not guarantee termination. However, in Iris, the wp is not a primitive assertion but is instead defined *in the logic*. This definition is not identical to the informal explanation of wp that we have just given; rather, that informal explanation is a consequence of the soundness theorem of the logic. We discuss the definition of the wp in §10. For now, we treat it as an abstract predicate that satisfies the rules in Figure 2.

The structural rule WpWand can be used to prove both monotonicity of the postcondition as well as the *frame rule* shown below.

$$\mathsf{wp}\ e\ \{\Phi\} * P \vdash \mathsf{wp}\ e\ \{v.\ \Phi(v) * P\}$$

The frame rule is the hallmark of separation logic and is the key ingredient to *modular* reasoning about programs: If we prove a specification wp $e\ \{\Phi\}$ for a program e, then any assertion P on some unrelated state (that is disjoint from the state used by the wp) will not affect the execution. As such, P can be passed along to the postcondition and used unchanged once e has finished executing.

The rule WpValue is the "base case" of the wp, since we just plug the value into the postcondition. The rule WpBind can be used to focus on a sub-expression in an evaluation context. All the remaining reasoning rules correspond to base reduction steps, with the rule WpPure reusing the pure reduction relation from Figure 1. The rules for the heap-manipulating steps are written in a "predicate transformer style" with Φ acting as the continuation. For example, the rule WpStore says that to reason about a store of w to location ℓ, we must own $\ell \mapsto v$ for some value v. In addition, we must show that the postcondition Φ follows from having $\ell \mapsto w$, the updated points-to that reflects the result of the store. From this rule, it is straightforward to derive an alternate "direct style" version that looks more like a typical Hoare triple: $\ell \mapsto v \vdash \mathsf{wp}\ \ell \leftarrow w\ \{v.\ \ulcorner v = ()\urcorner * \ell \mapsto w\}$.

There is no rule in Figure 2 for reasoning about loops as SeqLang does not have a primitive loop or recursion mechanism. However, since it is a higher-order language and supports higher-order state, recursion can be encoded in multiple ways. We postpone the discussion of how to reason about recursion until §2.3.

Before considering the completeness question, we verify two examples to demonstrate our logic.

Example 1. We prove that the program $40 + (1 + 1)$ evaluates to the value 42. We start with the goal at the top and apply the rules one-by-one, noting what is left to show. This is how the rules are intended to be used, and the resulting order of proof steps follows the program's execution order.

$$\begin{aligned}
& \vdash \mathsf{wp}\ 40 + (1 + 1)\ \{v.\ \ulcorner v = 42\urcorner\} \\
\Leftarrow\ & \vdash \mathsf{wp}\ 1 + 1\ \{w.\ \mathsf{wp}\ 40 + w\ \{v.\ \ulcorner v = 42\urcorner\}\} && \text{by WpBind with } K := 40 + \bullet \\
\Leftarrow\ & \vdash \mathsf{wp}\ 2\ \{w.\ \mathsf{wp}\ 40 + w\ \{v.\ \ulcorner v = 42\urcorner\}\} && \text{by WpPure} \\
\Leftarrow\ & \vdash \mathsf{wp}\ 40 + 2\ \{v.\ \ulcorner v = 42\urcorner\} && \text{by WpValue} \\
\Leftarrow\ & \vdash \mathsf{wp}\ 42\ \{v.\ \ulcorner v = 42\urcorner\} && \text{by WpPure} \\
\Leftarrow\ & \vdash \ulcorner 42 = 42\urcorner && \text{by WpValue}
\end{aligned}$$

Example 2. We prove that the program $(\lambda x.\ !x)\ (\mathbf{ref}(42))$ evaluates to the value 42 as well.

$$\begin{aligned}
& \vdash \mathsf{wp}\ (\lambda x.\ !x)\ (\mathbf{ref}(42))\ \{v.\ \ulcorner v = 42\urcorner\} \\
\Leftarrow\ & \vdash \mathsf{wp}\ \mathbf{ref}(42)\ \{w.\ \mathsf{wp}\ (\lambda x.\ !x)\ w\ \{v.\ \ulcorner v = 42\urcorner\}\} && \text{by WpBind} \\
\Leftarrow\ & \ell \mapsto 42 \vdash \mathsf{wp}\ \ell\ \{w.\ \mathsf{wp}\ (\lambda x.\ !x)\ w\ \{v.\ \ulcorner v = 42\urcorner\}\} && \text{by WpAlloc} \\
\Leftarrow\ & \ell \mapsto 42 \vdash \mathsf{wp}\ (\lambda x.\ !x)\ \ell\ \{v.\ \ulcorner v = 42\urcorner\} && \text{by WpValue} \\
\Leftarrow\ & \ell \mapsto 42 \vdash \mathsf{wp}\ !\ell\ \{v.\ \ulcorner v = 42\urcorner\} && \text{by WpPure}
\end{aligned}$$

$$\Leftarrow \qquad \ell \mapsto 42 \vdash \ulcorner 42 = 42 \urcorner \qquad \text{by WpLoad}$$

Soundness. We have seen how the logic can be used to verify simple programs, and we have discussed that the wp encodes a partial correctness criterion. However, we still need to show that the logic is sound: proving a wp about a program should establish partial correctness of the program at the meta-level. To state this formally, we first define a meta-level predicate $\mathit{safe}_\varphi(e, \sigma)$:

$$\mathsf{red}(e, \sigma) \triangleq \exists e', \sigma'.\ (e, \sigma) \to (e', \sigma')$$
$$\mathit{safe}_\varphi(e, \sigma) \triangleq \forall e', \sigma'.\ (e, \sigma) \to^* (e', \sigma') \implies (\exists v.\ e' = v \land \varphi(v)) \lor \mathsf{red}(e', \sigma')$$

The predicate $\mathit{safe}_\varphi(e, \sigma)$ says that executing e never gets stuck, and if it terminates in a value v then the value satisfies the predicate φ.

Remark 3. *If $\mathit{safe}_\varphi(e, \sigma)$, then either e is a value satisfying φ, or (e, σ) is reducible.*

Remark 4. *If $\mathit{safe}_\varphi(e, \sigma)$ and $(e, \sigma) \to (e_2, \sigma_2)$, then $\mathit{safe}_\varphi(e_2, \sigma_2)$.*

The following *soundness*[2] theorem connects the wp to the *safe* predicate.

Theorem 5 (Soundness)**.** *If $\vdash \mathsf{wp}\ e\ \{v.\ \ulcorner \varphi(v) \urcorner\}$, then $\mathit{safe}_\varphi(e, \sigma)$ for all σ.*

We do not prove soundness here and instead continue with completeness. Completeness is the converse of soundness, where we go from a safety proof at the meta-level to a proof of the corresponding wp:[3]

Theorem 6 (Completeness)**.** *If $\mathit{safe}_\varphi(e, \sigma)$ for all σ, then $\vdash \mathsf{wp}\ e\ \{v.\ \ulcorner \varphi(v) \urcorner\}$.*

The goal for the remainder of this section is to prove this theorem.

2.2 Sequential Completeness for Terminating Programs

Before proving Theorem 6, we will warm up by proving a weaker version that makes the additional assumption that e is strongly normalizing under the reduction relation $\to$ (written $\mathsf{SN}_{\to}$), *i.e.*, always terminating when run from any state. Additionally, as our proof is going to proceed by induction, it will be helpful to generalize the statement to get a better induction hypothesis. Thus, we prove the following theorem: if e is safe in a specific σ, then ownership of the points-to assertions for all locations in σ entails the corresponding wp.

Theorem 7 (Completeness from Strong Normalization)**.** *If $\mathit{safe}_\varphi(e, \sigma)$ and $\mathsf{SN}_{\to}(e, \sigma)$, then $\ast_{(\ell \leftarrow v) \in \sigma}\ \ell \mapsto v \vdash \mathsf{wp}\ e\ \{v.\ \ulcorner \varphi(v) \urcorner\}$.*

Proof. The proof is by induction on the evidence that (e, σ) is strongly normalizing. This means that we get to assume the induction hypothesis for all immediate successors (e', σ') of (e, σ). Formally, we have the following induction hypothesis:

$$\forall e', \sigma'.\ (e, \sigma) \to (e', \sigma') \Rightarrow \mathit{safe}_\varphi(e', \sigma') \Rightarrow \ast_{(\ell \leftarrow v) \in \sigma'}\ \ell \mapsto v \vdash \mathsf{wp}\ e'\ \{w.\ \ulcorner \varphi(w) \urcorner\} \qquad \text{(IH)}$$

and we are left to prove $\mathit{safe}_\varphi(e, \sigma) \Rightarrow \ast_{(\ell \leftarrow v) \in \sigma}\ \ell \mapsto v \vdash \mathsf{wp}\ e\ \{w.\ \ulcorner \varphi(w) \urcorner\}$.[4] Now, we just need to reason about (at least) one reduction step of e and then apply the induction hypothesis.

[2] In Iris, this theorem is usually called *adequacy* for reasons we discuss later in §10.

[3] Note that completeness of a program logic is different from the notion of semantic completeness traditionally found in logic, which says that if $\models P$ then $\vdash P$, where $\models P$ means that P holds in some class of semantic models. For program logics, the notion of semantics we are relating to is the *operational semantics* of the program, not the semantic model of assertions.

[4] As usual with this kind of induction on a well-founded reduction relation, there is no separate base case to consider.

We first appeal to Remark 3. If e is a value, we apply WpValue, which concludes the proof. In the remaining case, since we know our expression is reducible, we know that $e = K[e_1]$ for some K, e_1 and that $(e_1, \sigma) \to_{\mathbf{base}} (e_2, \sigma')$ for some e_2, σ'. Thus, using WpBind, we turn our goal into $\mathsf{wp}\ e_1\ \{v.\ \mathsf{wp}\ K[v]\ \{w.\ \ulcorner\varphi(w)\urcorner\}\}$. We continue with a case distinction on the base reduction.

Consider the case BasePure where $e_1 \to_{\mathbf{pure}} e_2$. We apply WpPure to turn our goal into $\mathsf{wp}\ e_2\ \{v.\ \mathsf{wp}\ K[v]\ \{w.\ \ulcorner\varphi(w)\urcorner\}\}$. Applying WpBind "backwards" turns our goal into $\mathsf{wp}\ K[e_2]\ \{w.\ \ulcorner\varphi(w)\urcorner\}$. Since $(e, \sigma) \to (K[e_2], \sigma)$, we instantiate our induction hypothesis with $(K[e_2], \sigma)$. By Remark 4, $(K[e_2], \sigma)$ is safe. Also, the state has not changed so we have the needed points-tos. Thus all premises of the induction hypothesis hold, finishing the case.

The four heap-dependent cases remain. In the BaseLoad case, we get that $e_1 = !\,\ell$, and that $\sigma(\ell) = v$. First, we remove the points-to for ℓ from the iterated separating conjunction over σ. Next, we use it to instantiate and apply WpLoad; the points-to for ℓ is returned in the postcondition. Ownership of the remaining locations is framed around the load. Finally, we re-establish the iterated separating conjunction for σ and finish the case by applying the induction hypothesis. For BaseStore, we do the same but with WpStore. Of course, the points-to now stores a new value, but σ is altered in lockstep, so we can still re-establish the iterated separating conjunction and apply the induction hypothesis. For BaseFree, we also remove the points-to for ℓ from the iterated separating conjunction, but WpFree does not return the points-to in the postcondition. Since it is also deleted from σ, we can still establish the iterated separating conjunction for the new state before applying the induction hypothesis.

Only the case BaseAlloc remains. Here, $e_1 = \mathbf{ref}(v)$ and $e_2 = \ell_1$ for some $\ell_1 \notin \mathrm{dom}(\sigma)$. When applying WpAlloc, we get a (fresh) non-deterministic location ℓ_2 along with a points-to, which might be different from ℓ_1. Since we own a points-to for all locations in σ, we use PointsToNe to establish that $\ell_2 \notin \mathrm{dom}(\sigma)$. Since $\ell_2 \notin \mathrm{dom}(\sigma)$ and our allocator is non-deterministic, we have that $(e_1, \sigma) \to_{\mathbf{base}} (\ell_2, \sigma[\ell_2 \leftarrow v])$ is a valid reduction step. Thus, we apply the induction hypothesis to $(K[\ell_2], \sigma[\ell_2 \leftarrow v])$ and finish the proof. □

Taking a step back, the structure of this proof exploits the fact that *either* e is already a value, and hence satisfies φ by assumption, or else e is reducible, in which case we perform a case distinction on the reduction rule, appealing to WpBind as needed. After applying a proof rule for the corresponding reduction rule, we use WpBind in the opposite direction to return to something that the induction hypothesis can be applied to. Below, we introduce an alternative induction principle that allows us to conduct a similar proof but *without* the strong normalization assumption.

2.3 Sequential Completeness for Partial Programs

Before we attempt to eliminate the strong normalization assumption from our proof, we explain how Iris-based logics typically allow us to reason about non-terminating programs. Consider the program $\Omega \triangleq (\lambda x.\, x\, x)\ (\lambda x.\, x\, x)$. It is easy to see that $\Omega \to_{\mathbf{pure}} \Omega$. Since our wp is intended to cover partial correctness, we should be able to prove the specification $\mathsf{wp}\ \Omega\ \{v.\ \mathsf{False}\}$. But how?

The core reasoning principle for recursion in typical Iris-based logics is *Löb induction*, which surfaces the underlying step-indexed nature of Iris [65, 69]. In the logic, this is exposed by means of the so-called *later modality* [3, 6, 57], written $\triangleright P$. Intuitively, the assertion $\triangleright P$ says that P is only "approximately" true now, but will be actually true after one step of computation.

The connection between program steps and the $\triangleright$ modality can be seen in the rules for the wp in Figure 2. For example, in the rule WpStore the assumption to the left of the turnstile is $\ell \mapsto v * \triangleright(\ell \mapsto w \mathrel{-\!\!*} \Phi(()))$. The left conjunct requires us to own the points-to for ℓ containing some value v before the store occurs. The right conjunct says that—one step later—after the store has happened, we get back $\ell \mapsto w$ (reflecting the updated value) and have to establish the postcondition.

$$\textsc{LaterIntro}\quad P \vdash \triangleright P \qquad \textsc{LaterMono}\quad \frac{P \vdash Q}{\triangleright P \vdash \triangleright Q} \qquad \textsc{Löb}\quad \frac{\triangleright P \vdash P}{\vdash P}$$

The later modality commutes with most logical connectives, except for the magic wand and the existential quantifier.[5] There are three key rules for working with the later modality: LaterIntro, LaterMono, and Löb. Intuitively, LaterIntro tells us that if something is true now, it remains true one step later. (In particular, this means the wp rules with $\triangleright$ imply the ones without $\triangleright$.) The rule LaterMono is the main elimination rule for the later modality: we can eliminate a later modality in our assumptions if we can remove a later modality from the conclusion as well.

The critical rule for reasoning about loops is Löb induction, which reifies induction on the step index: To prove P, it suffices to prove P under the induction hypothesis $\triangleright P$. This allows a coinductive style of reasoning: we can use the induction hypothesis $\triangleright P$ only after we have made "progress" by taking a step in the program, which lets us apply LaterMono to "unlock" the induction hypothesis.

Example 8. We demonstrate how Löb induction allows us to verify the program Ω:

$$\begin{array}{lll} & \vdash \mathsf{wp}\ \Omega\ \{v.\ \mathsf{False}\} & \\ \Leftarrow & \triangleright\, \mathsf{wp}\ \Omega\ \{v.\ \mathsf{False}\} \vdash \mathsf{wp}\ \Omega\ \{v.\ \mathsf{False}\} & \text{by Löb} \\ \Leftarrow & \Omega \rightarrow_{\mathbf{pure}} \Omega & \text{by WpPure} \end{array}$$

Here, applying Löb allows us to immediately conclude using WpPure. Standard rules for partial-correctness reasoning about recursive functions or while loops can be derived using Löb induction. The power of Löb induction is that it is not tied to a particular program construct but works for all propositions, and the induction hypothesis can have an arbitrary shape. This is useful for proving our completeness theorem.

Theorem 9 (Completeness of SeqLang). *If* $\mathit{safe}_\varphi(e, \sigma)$, *then* $\ast_{(\ell \leftarrow v) \in \sigma} \ell \mapsto v \vdash \mathsf{wp}\ e\ \{v.\ \ulcorner \varphi(v) \urcorner\}$.

The proof proceeds similarly to the proof of Theorem 7, but instead of doing induction on the fact that (e, σ) is strongly normalizing, we do Löb induction.[6] Instead of the requirement to exhibit an explicit step with the $\rightarrow$ relation, the induction hypothesis is now guarded by a later modality. But this is equivalent, since the later modality is the logic's way of requiring us to take a step! Indeed, the old proof continues to work. In all cases, we apply one of the wp rules which introduces a later modality in the goal. By applying LaterMono, we "unlock" the induction hypothesis and use it to conclude the proof.

3 Completeness for a Concurrent Language

We have seen how to prove completeness for an Iris-based program logic for a sequential language. In this section, we define the concurrent language ConcLang and show how our approach generalizes to this language.

3.1 A Concurrent Separation Logic

We begin by extending the grammar, reduction rules, and program logic with support for concurrency. These additions are outlined in Figure 3. On the language side, we add two new primitives. The first is **fork**, which implements unstructured concurrency. The second is an atomic compare-and-swap operation **CAS**, which allows us to write interesting concurrent programs. Threads are modeled using a thread-pool semantics, defined by the relation $\rightarrow_{\mathbf{tp}}$, which works on configurations ρ now storing a list of threads (expressions). We will treat this list as isomorphic to a finite partial function $\mathbb{N} \xrightarrow{\text{fin}} \mathit{Expr}$ mapping n to the element at index n in the list. To create new threads, the

[5] It commutes with the existential quantifier if the domain of quantification is non-empty.

[6] This requires that we move the assumption $\mathit{safe}_\varphi(e, \sigma)$ into the logic as the assertion $\ulcorner \mathit{safe}_\varphi(e, \sigma) \urcorner$.

$$e \in \mathit{Expr} ::= \cdots \mid \mathbf{fork}(e) \mid \mathbf{CAS}(e_1, e_2, e_3), \quad \vec{e} \in \mathcal{L}(\mathit{Expr}), \quad \rho \in \mathit{Conf} \triangleq \mathcal{L}(\mathit{Expr}) \times \mathit{State}$$

$$P, Q \in \mathit{iProp} ::= \cdots \mid \mathsf{wp}_{\mathcal{E}}\, e\, \{v.\, \Phi(v)\} \mid \Rrightarrow_{\mathcal{E}} P \mid \boxed{P}^{\mathcal{N}} \mid \bullet^{\gamma} m \mid k \hookrightarrow^{\gamma} v$$

TpStep
$$\frac{(T(n), \sigma) \to (e_2, \sigma', \vec{e}_f)}{(T, \sigma) \to_{\mathbf{tp}} (T[n \leftarrow e_2] +\!\!+ \vec{e}_f, \sigma')}$$

BaseFork
$$(\mathbf{fork}(e), \sigma) \to_{\mathbf{base}} ((), \sigma, [e])$$

InvAlloc
$$P \vdash \Rrightarrow_{\mathcal{E}} \boxed{P}^{\mathcal{N}}$$

UpdateInv
$$\frac{\mathcal{N} \subseteq \mathcal{E} \qquad P * Q \vdash \Rrightarrow_{\mathcal{E} \setminus \mathcal{N}} P * R}{\boxed{P}^{\mathcal{N}} * Q \vdash \Rrightarrow_{\mathcal{E}} R}$$

WpAtomicInv
$$\frac{\mathcal{N} \subseteq \mathcal{E} \qquad \text{Atomic } e \qquad P \vdash \mathsf{wp}_{\mathcal{E} \setminus \mathcal{N}}\, e\, \{v.\, P * \Phi(v)\}}{\boxed{P}^{\mathcal{N}} \vdash \mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}}$$

WpUpdateElim
$$\frac{P \vdash \mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}}{\Rrightarrow_{\mathcal{E}} P \vdash \mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}}$$

WpWand
$$\mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\} * (\forall x.\, \Phi(x) \mathrel{-\!\!*} \Rrightarrow_{\mathcal{E}} \Psi(x)) \vdash \mathsf{wp}_{\mathcal{E}}\, e\, \{\Psi\}$$

WpValue
$$\Rrightarrow_{\mathcal{E}} \Phi(v) \dashv\vdash \mathsf{wp}_{\mathcal{E}}\, v\, \{\Phi\}$$

WpFork
$$\triangleright \Rrightarrow_{\mathcal{E}} (\Phi(()) * \mathsf{wp}_{\top}\, e\, \{v.\, \mathsf{True}\}) \vdash \mathsf{wp}_{\mathcal{E}}\, \mathbf{fork}(e)\, \{\Phi\}$$

Fig. 3. Syntax, semantics, and a concurrent separation logic for ConcLang.

relations $\to$ and $\to_{\mathbf{base}}$ have a new parameter $\vec{e}_f$, which is a list of forked-off expressions. This list is empty in all rules except BaseFork, where it is a singleton list since **fork** creates one new thread.

On the program logic side, the important new rule is WpFork, which allows us to spawn a new thread. Applying this rule lets us reason about concurrent threads by dividing our resources between the current thread and the new thread. For example, to establish $\mathsf{wp}\ \mathbf{fork}(e_1); e_2\ \{\Phi\}$, we apply WpBind and WpFork and are left with the goal $\mathsf{wp}\ e_1\ \{v.\, \mathsf{True}\} * \mathsf{wp}\ (); e_2\ \{\Phi\}$ (ignoring $\Rrightarrow$ and $\mathcal{E}$ for now). That is, we must prove two *separate* wps, one for each thread. This is a key power of concurrent separation logic: We can verify threads in isolation, as long as we can separate the assumptions needed to verify each of them. However, the disjoint separation means that if we have a points-to assertion $\ell \mapsto v$, we can only give it to one of the two threads. That suffices if the two threads access disjoint pieces of state, but if they need to share access to the same state, then we need other features in the logic: *invariants* and *ghost state*.

Invariants and Masks. If we want two threads to operate on the same heap cell, we need to somehow give both of them access to it. Since the executions of both threads interleave, we can no longer reason about each thread in complete isolation: we need a way to describe the *protocol* that the two threads use to cooperate. In Iris, this is achieved using the invariant assertion $\boxed{P}^{\mathcal{N}}$, which says that P is an invariant of the program execution, *i.e.*, it holds at all times. Invariants are annotated with an identifying *namespace* $\mathcal{N}$. For bookkeeping purposes, the wp assertion is annotated with a *mask* $\mathcal{E}$ that determines the invariant names that a specification may rely on. We use $\top$ to denote the set of all names. Together, namespaces and masks are used to prevent the prover from opening the same invariant twice in a nested fashion, which would be unsound. The technical details of namespaces and masks are orthogonal to our focus; see Jung et al. [36] for the full exposition.

Invariants are freely duplicable,[7] *i.e.*, $\boxed{P}^{\mathcal{N}} \vdash \boxed{P}^{\mathcal{N}} * \boxed{P}^{\mathcal{N}}$, and thus shareable across threads. This is sound because an invariant assertion merely conveys the *knowledge* that an invariant is maintained. A thread may use $\boxed{P}^{\mathcal{N}}$ to access P for *one atomic step*, so long as it re-establishes P after that step is completed.[8] This process is captured by the rule WpAtomicInv. After opening an invariant, the invariant's namespace is removed from the mask. Additionally, we have a side condition Atomic e requiring that e is *atomic*, *i.e.*, it reduces to a value in a single reduction step.

Example 10. Let us now verify the following specification about a program which shares a location between two threads.[9]

$$\mathrm{wp}_{\top}\ \mathbf{let}\ x = \mathbf{ref}(0)\ \mathbf{in}\ \mathbf{fork}\ (x \leftarrow 1); !x\ \left\{v.\ \ulcorner v \in \{0,1\}\urcorner\right\}$$

The proof proceeds by first allocating the location ℓ_x. Next, we can allocate an invariant using InvAlloc (keep ignoring the $\Rrightarrow$ for now). The invariant we allocate is $\boxed{\exists v \in \{0,1\}.\ \ell_x \mapsto v}^{\mathcal{N}}$ for some $\mathcal{N}$. The choice of $\mathcal{N}$ does not matter in this example. Since invariant assertions are duplicable, we can pass it to both threads when we apply WpFork. Both threads will now proceed to access the invariant. For the forked-off thread, we use WpAtomicInv and since a store operation is atomic, we get access to the points-to for the duration of the operation. Afterwards, we restore the invariant. This is possible since we store 1, and $1 \in \{0,1\}$. Similarly, when performing the load in the main thread, we open the invariant to learn $v \in \{0,1\}$, which establishes the postcondition.

To summarize, we have created an invariant, shared it between two threads, and opened it around atomic operations in each thread. The choice of the invariant was up to us, but we have to ensure that all threads maintain it. This is a typical pattern for concurrent program verification in Iris.

Ghost State and Resource Updates. While the invariants we just presented are sufficient for reasoning about simple concurrent programs, it has long been known [59] that for general reasoning about concurrent programs, we need additional *auxiliary* or *ghost* state.

The *update modality* $\Rrightarrow_{\mathcal{E}}$ is the primary primitive for manipulating ghost resources in Iris. Intuitively, the assertion $\Rrightarrow_{\mathcal{E}} P$ denotes a resource that can be *changed* to a resource satisfying P. The way a resource may change is resource-specific; we will explain the permitted changes for the resource relevant to this proof below. This update may make use of the invariants in mask $\mathcal{E}$, but must also promise to maintain those invariants (UpdateInv). The update modality can be eliminated at any suitable time during program verification using WpUpdateElim. The update modality also shows up in the invariant allocation rule InvAlloc, as that rule has to register the new invariant in the ghost state.

Iris provides a general mechanism for users to define their own custom ghost state; we refer to Jung et al. [36] for the details. For this paper, however, we will only need *ghost map* assertions that can be treated as abstract predicates governed by the rules shown below.

GhostMapLookup
$$\bullet^{\gamma} m * k \hookrightarrow^{\gamma} v \vdash \ulcorner m(k) = v \urcorner$$

GhostMapAlloc
$$\vdash \Rrightarrow_{\mathcal{E}} \exists \gamma.\ \bullet^{\gamma} \varnothing$$

GhostMapInsert
$$\frac{k \notin \mathrm{dom}(m)}{\bullet^{\gamma} m \vdash \Rrightarrow_{\mathcal{E}} \bullet^{\gamma} m[k \leftarrow v] * k \hookrightarrow^{\gamma} v}$$

GhostMapUpdate
$$\bullet^{\gamma} m * k \hookrightarrow^{\gamma} v \vdash \Rrightarrow_{\mathcal{E}} \bullet^{\gamma} m[k \leftarrow v'] * k \hookrightarrow^{\gamma} v'$$

GhostMapDelete
$$\bullet^{\gamma} m * k \hookrightarrow^{\gamma} v \vdash \Rrightarrow_{\mathcal{E}} \bullet^{\gamma} (m \setminus \{k\})$$

[7] Using Iris terminology, the invariant assertion is *persistent*, which allows us to use it several times.
[8] For soundness reasons, invariants only give access to $\triangleright P$. In this paper, our invariants are always *timeless*, so we can eliminate this later modality. See Jung et al. [36] for the details and a definition of timelessness.
[9] The program uses let-bindings and sequencing, which are easily encoded using lambdas.

Ghost maps consist of two types of assertions: the *authoritative* assertion $\bullet^\gamma m$ and a "ghosts-to" assertion $k \hookrightarrow^\gamma v$. Both assertions are indexed by a ghost name γ that identifies a particular ghost map instance. The $\bullet^\gamma m$ assertion tracks the full or "authoritative" view of the ghost map m with name γ. A new ghost map can always be allocated with a fresh ghost name (GhostMapAlloc). The ghosts-to $k \hookrightarrow^\gamma v$ tracks a partial view of the ghost map with name γ and that the key k currently stores the value v. That is, by owning $\bullet^\gamma m$ and $k \hookrightarrow^\gamma v$ one may conclude $m(k) = v$ (GhostMapLookup). New entries can be inserted (GhostMapInsert), updated (GhostMapUpdate), and deleted (GhostMapDelete) accordingly. Since these change ghost state, they are guarded by the update modality.

3.2 Completeness for Concurrent Programs

Now that we have extended the logic with concurrency primitives, we revisit the question of soundness and completeness. First, our soundness result changes due to the switch to a thread-pool semantics, which requires us to state safety at the level of configurations:

$$\begin{aligned} \mathit{safe\text{-}tp}_\varphi(\vec{e}, \sigma) \triangleq \forall\, \vec{e}\,', \sigma'.\ (\vec{e}, \sigma) \to^*_{\mathtt{tp}} (\vec{e}\,', \sigma') \implies \\ \forall n, e''.\ \vec{e}\,'(n) = e'' \Rightarrow (\exists v.\ e'' = v \wedge (n = 0 \Rightarrow \varphi(v))) \vee \mathsf{red}(e'', \sigma') \\ \mathit{safe}_\varphi(e, \sigma) \triangleq \mathit{safe\text{-}tp}_\varphi([e], \sigma) \end{aligned}$$

Intuitively, no thread may get stuck, but only the "main" thread, the first in $\vec{e}\,'$, has to satisfy the postcondition φ on termination.[10] For this modified definition, we have the following analogues of Remark 3 and Remark 4.

Remark 11. *If $\mathit{safe\text{-}tp}_\varphi(\vec{e}, \sigma)$, then either $\vec{e}(n)$ is a value, satisfying φ if $n = 0$; or $(\vec{e}(n), \sigma)$ is reducible.*

Remark 12. *If $\mathit{safe\text{-}tp}_\varphi(\vec{e}, \sigma)$ and $(\vec{e}, \sigma) \to_{\mathtt{tp}} (\vec{e}_2, \sigma_2)$, then $\mathit{safe\text{-}tp}_\varphi(\vec{e}_2, \sigma_2)$.*

The soundness theorem establishes safety for a thread pool with e as the initial main thread.

Theorem 13 (Soundness). *If $\vdash \mathsf{wp}_\top\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\}$, then $\mathit{safe}_\varphi(e, \sigma)$ for all σ.*

For the completeness theorem, recall the two main facts that were maintained inductively in our previous sequential proof: (1) the current expression e and state σ are *safe*, and (2) we have points-tos for all the locations in σ. As we move to the concurrent setting, safety is a property of the entire thread pool, and the points-tos for the locations may be shared across threads. To deal with these changes, we first introduce ghost state to track the status of the concurrently running threads. Specifically, we use a ghost map of type $\mathbb{N} \xrightarrow{\mathsf{fin}} \mathit{Expr}$ to track the thread pool. This way, even though we reason about one thread at a time when proving a wp, we maintain that the overall system is *safe* at all times. Each thread owns the ghosts-to for its own thread identifier. Second, in order to allow threads to share access to the state, we put all the heap points-tos in an invariant, along with the authoritative copy of the thread pool ghost state. This invariant looks as follows.

$$I_{compl} \triangleq \boxed{\exists\, \vec{e}, \sigma.\, \ulcorner \mathit{safe\text{-}tp}_\varphi(\vec{e}, \sigma) \urcorner * \bullet^\gamma \vec{e} * \textstyle\ast_{(\ell \leftarrow v) \in \sigma} \ell \mapsto v}^{\mathcal{N}}$$

We can now state the core lemma we are going to prove by Löb induction.

Lemma 14 (Per-thread Completeness). $I_{compl} * n \hookrightarrow^\gamma e \vdash \mathsf{wp}_\top\, e\, \{v.\, n \hookrightarrow^\gamma v\}$.

As in the sequential setting, the proof of this lemma eliminates the safety assumption to deduce that e is either a value or reducible; in the latter case, we perform a case analysis on which step it takes. Since the safety assumption is maintained by the invariant, we first open the invariant.

[10] We think of the overall return value of the concurrent execution as being the value that the first thread returns. This also matches the way forked-off threads have the postcondition True in WpFork.

This immediately presents us with an issue: We can only keep the invariant open around the next step if e is atomic, but without opening the invariant we do not know whether e is atomic or not. Therefore, we do not know whether to apply WpAtomicInv or UpdateInv. The existing rules do not easily allow us to open the invariant and only then make the decision of whether to keep it open for the next step or not. To overcome this issue, we need Lemma 15.

Lemma 15. *If $\mathcal{N} \subseteq \mathcal{E}$, and if e is reducible under some σ, then it holds that*

$$\begin{aligned}&\boxed{P}^{\mathcal{N}} * \left(P \mathrel{-\!\!*} \Rrightarrow_{\mathcal{E}\setminus\mathcal{N}}\right.\\ &\qquad \left(\exists K, e'.\ \ulcorner e = K[e']\urcorner * \ulcorner \mathsf{Atomic}\, e'\urcorner * \left(\exists \sigma'.\ \ulcorner \mathsf{base\text{-}red}\,(e', \sigma')\urcorner\right) *\right.\\ &\qquad\quad \left.\left.\mathsf{wp}_{\mathcal{E}\setminus\mathcal{N}}\, e'\, \{v.\, P * \mathsf{wp}_{\mathcal{E}}\, K[v]\, \{\Psi\}\}\right) \vee\right.\\ &\qquad \left.P * \mathsf{wp}_{\mathcal{E}}\, e\, \{\Psi\}\right)\\ &\vdash \mathsf{wp}_{\mathcal{E}}\, e\, \{\Psi\}\end{aligned}$$

This lemma lets us delay this choice until after the invariant has been opened. In particular, with this rule we can prove Atomic e *after* opening the invariant, using assumptions previously kept in the invariant.[11] At its heart, what the lemma says is that *after* obtaining the contents P of the invariant, we can decide to either prove that e is atomic and close the invariant after taking a step (left disjunct) or immediately close the invariant again (right disjunct).

Returning to the proof of Lemma 14: using Lemma 15, we can open the invariant and use GhostMapLookup and Remark 11 to learn that e is either a value or it is reducible.[12] If e is a value, we conclude with WpValue. If not, we derive $e = K[e_1]$ where e_1 is base-reducible. That is, we get that $(e_1, \sigma) \to_{\mathbf{base}} (e', \sigma', \vec{e}_f)$ for some $\sigma, e', \sigma', \vec{e}_f$. We perform case distinction on the base step.

For *pure steps* $e_1 \to_{\mathbf{pure}} e'$, we have to immediately close the invariant again (*i.e.*, we pick the right disjunct in Lemma 15), since pure steps are in general not atomic. While this forces us to give up all assumptions about the state, we do not actually need them, since pure steps do not care about the state. Using WpBind and WpPure, we are left with proving $\mathsf{wp}_{\top}\, K[e']\, \{\cdots\}$. We also unlocked the Löb induction hypothesis since the pure step allowed us to eliminate a later. But to apply the induction hypothesis, we would need $n \hookrightarrow^{\gamma} K[e']$ while we currently only have $n \hookrightarrow^{\gamma} K[e_1]$. We can update our ghosts-to by opening the invariant again (for 0 steps) and using GhostMapUpdate. To then close the invariant again, we use Remark 12 to prove that the new configuration is safe.

For BaseFork, we have $e_1 = \mathbf{fork}\, e_f$. Here, we also close the invariant (right disjunct of Lemma 15) and proceed similarly to the previous case. But now we need to prove two wp assertions, one for each thread. To do so, we can again open the invariant to update $n \hookrightarrow^{\gamma} K[\mathbf{fork}\, e_f]$ to $K[()]$, but also insert a new entry n' (using GhostMapInsert) to create a new ghosts-to $n' \hookrightarrow^{\gamma} e_f$ for the newly created thread. We close the invariant again. We now use our Löb induction hypothesis *twice*, once for the parent thread and once for the new thread. This is possible because Löb induction hypotheses hold without any resources and are thus duplicable.

For *atomic expressions* modifying memory, we can keep the invariant open (left disjunct of Lemma 15). This lets us obtain ownership of the entire heap (the iterated separating conjunction with all points-tos), so we can proceed like in the sequential case. Handling **CAS** is also straightforward since it is very similar to a load or a store. Once this is done, we close the invariant, which once again requires updating the ghosts-tos like in the other cases.

[11] Our proof of this lemma makes use of the excluded middle in the meta-logic. This could be avoided by proving that Atomic e is decidable.

[12] Lemma 15 already requires that e is reducible, so we actually open the invariant (for 0 steps, using UpdateInv) already once before applying the lemma.

This lemma, while having a rather technical precondition, contains the "meat" of the completeness proof. Proving a lemma that is the obvious inverse of soundness is now easy:

Theorem 16 (Completeness of ConcLang). *If $\mathit{safe}_\varphi(e, \sigma)$ for all σ, then* $\vdash \mathsf{wp}_\top\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\}$.

Proof. We instantiate the precondition with the empty heap $\emptyset$ to get $\mathit{safe\text{-}tp}_\varphi([e], \emptyset)$. Next, we use GhostMapAlloc to get an empty ghost map at γ, and insert $0 \leftarrow e$ into it giving us $0 \hookrightarrow^\gamma e$. We allocate the invariant I_{compl} since we have all required resources, including the iterated separating conjunction over the empty set. From Lemma 14, we get $\mathsf{wp}_\top\, e\, \{v.\, 0 \hookrightarrow^\gamma v\}$, while we must derive $\mathsf{wp}_\top\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\}$. For this, we use WpWand, which also allows us to open invariants with UpdateInv. We assume $0 \hookrightarrow^\gamma v$ and must show $\Rightarrow\!\!\Vdash_\top \ulcorner \varphi(v) \urcorner$, for which we once again open I_{compl}, and then use GhostMapLookup and Remark 11. Since v is a value and thus not reducible, and since $n = 0$, we get $\varphi(v)$, which finishes the proof. □

3.3 Language-Independent Completeness

So far, we considered proving completeness for a *concrete* language. However, Iris provides a default program logic based on a *generic* wp construction, suitable for any language Λ that implements the *language interface*. This wp comes with a language-independent soundness theorem that factors most of the work of proving soundness for a particular language into a reusable building block. This raises the question: is there a similar building block for completeness?

The language interface requires Λ to have a concept of expressions e, values v, state σ and evaluation contexts K. It must also have contextual operational semantics derived from a base reduction rule $\rightarrow_{\mathbf{base}}$, with concurrency handled via thread-pool semantics.

To determine what a completeness recipe for an arbitrary instance of this interface could look like, we return to the proof of Theorem 16 and split it into a generic and a language-specific part. On the language-specific side, we have the individual cases for each base reduction step, and the fact that we need an iterated separating conjunction over the entire state σ (which can look very different for other languages). To abstract over the iterated separating conjunction, we introduce a predicate $\mathsf{S}_\circ : \Lambda.\mathit{State} \to \mathit{iProp}$ collecting all language-specific resources required to represent the state. To abstract over the base reduction case distinction, we formulate a general condition on base reductions that is sufficient to obtain completeness. Our language-independent completeness theorem then only requires that Λ satisfies this condition.

In the following, we discuss a simplified version of language-independent completeness for languages such as HeapLang. A more general form that can also handle λ_{Rust} is shown in Appendix A.

Theorem 17 (Language-Independent Completeness). *Let Λ be a language satisfying Condition 18 given below. If $\mathit{safe}_\varphi(e, \sigma)$, then* $\mathsf{S}_\circ(\sigma) \vdash \mathsf{wp}_\top\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\}$.

Condition 18 (Completeness).

$$\ulcorner \mathsf{base\text{-}red}(e, \sigma) \urcorner * \mathsf{S}_\circ(\sigma) \vdash \Rightarrow\!\!\Vdash_{\mathcal{E}} \left(\ulcorner \mathsf{Atomic}\, e \urcorner * \forall \Phi.\, \triangleright\, \mathit{WithStatePre}(\mathcal{E}, e, \sigma, \Phi) \mathrel{-\!\!*} \mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}\right) \vee \left(\mathsf{S}_\circ(\sigma) * \forall \Phi.\, \triangleright\, \mathit{WithoutStatePre}(\mathcal{E}, e, \Phi) \mathrel{-\!\!*} \mathsf{wp}_\top\, e\, \{\Phi\}\right)$$

where

$$\mathit{WithStatePre}(\mathcal{E}, e, \sigma, \Phi) \triangleq \forall v', \sigma', \vec{e}_f.\, \ulcorner (e, \sigma) \rightarrow (v', \sigma', \vec{e}_f) \urcorner * \mathsf{S}_\circ(\sigma') \mathrel{-\!\!*} \Rightarrow\!\!\Vdash_{\mathcal{E}} \Phi(v') * \mathop{\ast}_{e_f \in \vec{e}_f} \mathsf{wp}_\top\, e_f\, \{v.\, \mathsf{True}\}$$

$$\mathit{WithoutStatePre}(\mathcal{E}, e, \Phi) \triangleq \forall e', \vec{e}_f.\, \left(\forall \sigma'.\, \mathsf{S}_\circ(\sigma') \mathrel{-\!\!*} \Rightarrow\!\!\Vdash_{\mathcal{E}} \exists \sigma''.\, \ulcorner (e, \sigma') \rightarrow^+ (e', \sigma'', \vec{e}_f) \urcorner * \mathsf{S}_\circ(\sigma'')\right) \mathrel{-\!\!*} \Rightarrow\!\!\Vdash_\top \mathsf{wp}_\top\, e'\, \{\Phi\} * \mathop{\ast}_{e_f \in \vec{e}_f} \mathsf{wp}_\top\, e_f\, \{v.\, \mathsf{True}\}$$

To understand this condition, imagine we are trying to prove it for some language Λ, and ignore the grayed-out parts for now (they are only relevant for reduction steps that fork new threads). We get to assume that e is base-reducible, as well as $\mathsf{S}_\circ(\sigma)$. Our proof can now go ahead by case distinction on base-reducibility. The condition offers us two choices for proceeding.

If we need access to $\mathsf{S}_\circ$ to execute e, we proceed with the left disjunct by proving $\mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}$ for some arbitrary Φ, under the *WithStatePre* precondition. This is only possible if e is atomic. Proving this will require a language-specific rule, and since e is atomic, this rule should just leave us with $\Phi(v')$ for some v'. The only way to prove $\Phi(v')$ is to use *WithStatePre*. That means we have to prove that (e, σ) actually steps to (v', σ') and that we can update $\mathsf{S}_\circ(\sigma)$ to $\mathsf{S}_\circ(\sigma')$.

If we do not need to keep $\mathsf{S}_\circ$ around for the execution of e, we proceed with the right disjunct. In this case, we must give up $\mathsf{S}_\circ(\sigma)$. Our goal is now $\mathsf{wp}_{\top}\, e\, \{\Phi\}$ with the mask $\top$. Again, the next step is to apply some language-specific proof rule. The idea is that this rule makes progress on the proof by reducing e to e' (think of WpPure). As long as we take at least one step in this reduction, we can apply *WithoutStatePre* to obtain a wp for e'. More specifically, applying *WithoutStatePre* forces us to prove that we can reduce e to e' starting in *any* state σ', potentially different from the σ we encountered before, and potentially changing it to σ''.[13] As part of this, we are given $\mathsf{S}_\circ(\sigma')$ and have to update it to $\mathsf{S}_\circ(\sigma'')$. If we can prove such a state-independent reduction from e to e', then in exchange we obtain a wp for e'.

The grayed-out parts handle fork-based concurrency: If any steps we take fork off new threads $\vec{e}_f$, then we need wp assertions not just for e' (or the postcondition for v') but also the new threads.

Theorem 17 (or rather, its generalization using Condition 32) is powerful enough to verify many of our case studies. A similar theorem can be proven for sequential languages; this gives rise to a slightly stronger conclusion (see Appendix B). Some case studies come with a custom definition of wp which makes Theorem 17 formally inapplicable; for these we are able to re-prove a very similar theorem by closely following the pattern outlined so far.

4 Case Study: Completeness of HeapLang

Having demonstrated the structure of our approach, we turn our attention to proving completeness of existing Iris-based program logics. We begin with HeapLang, the built-in language of Iris which is often used as a starting point for other program logics. HeapLang is very similar to ConcLang, with two main differences. First, freeing a heap location leaves behind a tombstone that prevents this location from being re-allocated in the future. Second, HeapLang has support for prophecy variables [37]. The full definition and program logic for HeapLang can be found in Appendix D.

The first point is relevant for completeness because it means a program can assert that two separately allocated locations are different even if the first location is freed before the second one is allocated. HeapLang did not come with a proof rule intended to cover this case, so it may seem like we cannot prove the correctness of such a program and hence the logic is not complete. However, it turns out that the existing rules for HeapLang's "metadata" mechanism (which allows the proof to associate arbitrary ghost data with a physical location) are sufficient to derive correctness of that program. By adding "metadata" tokens in $\mathsf{S}_\circ$ and adding a new law for metadata token exclusivity, we are able to establish Condition 18.

Prophecy variables are technically challenging because of how they are embedded in the operational semantics via two "virtual" operations: **`newproph`** creates a new prophecy variable with a unique identifier; **`resolve`** p **`to`** v[14] resolves prophecy variable p to value v by adding a special

[13] In the example we considered above, the state will never change here, but in §6 we will see an example of a language where the "without-state" case is useful for state-changing reductions. Similarly, our examples have so far only taken one step whereas the condition allows taking multiple; this is a straightforward generalization of Remark 12 and also used in §6.
[14] This is a simplified version, see Appendix D for the general version.

observation label to the base step. To ensure freshness of prophecy variables, the state now carries two components: $\sigma.h$ is the usual heap, and $\sigma.pid$ is the set of already used prophecy identifiers.

$$\textsc{BaseNewProph}\ \frac{p \notin \sigma.pid \qquad \sigma' = \sigma \text{ with } pid := \sigma.pid \cup \{p\}}{(\mathbf{newproph}, \sigma) \to_{\mathbf{base}} (p, \sigma', \epsilon)} \qquad \textsc{BaseResolve}\ \frac{p \in \sigma.pid}{(\mathbf{resolve}\ p\ \mathbf{to}\ v, \sigma) \xrightarrow{[(p,v)]}_{\mathbf{base}} ((), \sigma, \epsilon)}$$

As part of this work, we had to slightly adjust BaseResolve (highlighted in blue): the original rule did not have a premise, *i.e.*, **resolve** p **to** v would reduce even if p did not exist. Since the reasoning rule for **resolve** requires ownership of p, the missing premise made the logic incomplete. After adding the missing precondition (a change that has been accepted by the Iris project), we can show Condition 18.[15]

Overall, this case study shows that our approach to proving completeness has the expected effect of identifying examples of programs that are correct with respect to the operational semantics, but cannot be verified with the existing proof rules. With only minor tweaks to the language and the logic, we establish completeness of HeapLang using Theorem 17.

5 Case Study: Completeness of a Total Correctness Logic

Apart from the wp $e\ \{\Phi\}$ assertion for partial correctness, Iris also comes with a variant wp $e\ [\Phi]$ for total correctness [43]. This new connective satisfies the following soundness theorem.

Theorem 19 (Total Soundness). *If* $\vdash \mathsf{wp}_\top\ e\ [v.\ulcorner\varphi(v)\urcorner]$, *then* $\mathit{safe}_\varphi(e,\sigma) \land \mathsf{SN}_{\to_{\mathbf{tp}}}([e],\sigma)$ *for all* σ.

This total wp connective is concurrent and proves *scheduler-independent* termination, requiring the program to terminate even under a maximally demonic scheduler.[16] The rules for the total wp are very similar to the ones discussed so far. The only difference is that none of the rules contain any later modalities, so that we are not allowed to eliminate a later modality when taking a step. This makes Löb induction impossible, forcing us to prove that programs actually terminate, typically by induction in the meta-level logic.

This also means that we cannot use Löb induction in our completeness proof either. Instead, we use the strong normalization assumption, echoing §2.2. Our proof mostly follows the general recipe of §3.3, and can be split into a language-specific total completeness condition and a language-independent part. In fact, the total completeness condition is identical to Condition 32, except that partial wps are substituted by total wps and later modalities are removed.

Theorem 20 (Language-Independent Total Completeness). *Let* Λ *be a language that satisfies the total completeness condition. If* $\mathit{safe}_\varphi(e,\sigma) \land \mathsf{SN}_{\to_{\mathbf{tp}}}([e],\sigma)$, *then* $\mathsf{S}_\circ(\sigma) \vdash \mathsf{wp}_\top\ e\ [v.\ulcorner\varphi(v)\urcorner]$.

The heart of the proof is a thread-local lemma akin to Lemma 14. The challenge in this proof is that it has to be done by induction on the strong normalization of some configuration $\rho_l = (\vec{e}_l, \sigma_l)$. But since all our knowledge about the "current" configuration is held inside the invariant, we do not have this "current" configuration available to do our induction. It seems like we need to open the invariant *before* starting the induction, but this does not lead to a sufficient induction hypothesis.

The solution is to do induction on a *lower bound* of the current configuration. Since the current configuration is not accessible at the start of the proof, we use additional ghost state to track past configurations that can reach the current configuration, one of which will be ρ_l. The proof is then by strong induction on this lower bound ρ_l being strongly normalizing.

[15] We also had to slightly strengthen the proof rule for **resolve** in a way that is orthogonal to this change, and related to the part of **resolve** that we simplified away for this discussion. See the **resolve** rule in Appendix D for details.
[16] See [43, p. 11] for a discussion on the limitations of this definition.

As such, our induction hypothesis holds for all (transitive) successors of ρ_l. Once we open the invariant, we learn (from the additional ghost state) that $\rho_l \to^*_{\mathbf{tp}} (\vec{e}, \sigma)$, where $(\vec{e}, \sigma)$ is the "current" configuration. As usual in our completeness proofs, we need to take a step $(\vec{e}, \sigma) \to_{\mathbf{tp}} (\vec{e}\,', \sigma')$ to a new configuration $(\vec{e}\,', \sigma')$. But by transitivity, this is also reachable from the lower bound ρ_l, so the induction hypothesis is applicable and we conclude the proof.

We instantiate this general recipe with the total correctness logic for HeapLang without prophecy variables (which are not supported by Iris's total correctness logic).

6 Case Study: Completeness of λ_{Rust}

RustBelt [35] develops a logical relation for a Rust-like type system. As part of this, they define a core calculus called λ_{Rust} that captures the central aspects of Rust, along with a program logic that is used to define the logical relation. We have applied our methodology to their calculus, uncovering some missing proof rules and, more interestingly, showing that the existing proof rules for memory accesses are complete for the non-standard memory model of this language.

λ_{Rust} is superficially similar to ConcLang. A complete grammar of λ_{Rust} is given in Appendix E. The main difference is the memory model, which distinguishes between atomic and non-atomic accesses, and is defined in a way that causes programs with data races to get stuck. We say a data race occurs when there are two concurrent accesses to the same location, in which at least one is a write and at least one is non-atomic. In a semantics where data races cause the execution to get stuck, a proof of safety (*i.e.*, a proof showing that a program never gets stuck) implies that the program is data-race-free. To detect data races in the semantics, λ_{Rust} essentially equips every memory location with a reader-writer lock. More precisely, a non-atomic access takes two steps: the first step acquires the lock, getting stuck if this is not possible; this produces an administrative redex. The second step performs the desired operation and releases the lock. Atomic accesses ensure the lock is not held in a conflicting way; they only take a single step.

The program logic of λ_{Rust} entirely hides the existence of these reader-writer locks. The points-to assertion $\ell \mapsto v$ ensures that the lock is currently not taken. λ_{Rust} also supports fractional permissions [7], with $\ell \mapsto_q v$ for $q < 1$ allowing for the existence of concurrent readers.[17] With this setup, the proof rules for loads and stores are entirely standard. In particular, the rules for atomic and non-atomic loads and stores are *the same*! However, only atomic accesses are Atomic, so WpAtomicInv cannot be used with non-atomic accesses.

Proving Completeness of the Data-Race-Free Program Logic. The program logic of λ_{Rust} is an instantiation of the language-independent wp construction that comes with Iris. To establish completeness, we would therefore like to appeal to the language-independent completeness theorem (Theorem 17), which requires us to pick a $S_\circ$ predicate. In the ConcLang and HeapLang instantiations, $S_\circ$ requires exclusive ownership of the points-tos for the *entire* heap. This poses an issue for λ_{Rust}: in proving Condition 18, we are forced to pick the second ("without-state") disjunct for the case of a non-atomic memory access. This requires us to give back $S_\circ$. We also need to keep ownership of the points-to for the location we are about to access, so we have to be able to give back $S_\circ$ without giving up the points-to. In the specific case of a non-atomic store, however, there is no reason why the points-to *must* be tracked by $S_\circ$: No other thread will access this location, because there would otherwise be a data race, which would contradict the assumption that the program is safe. Similarly, for reads, we know that other threads will not perform writes, so, intuitively, we should soundly be able to take a fraction of the points-to out of $S_\circ$.

[17] In fact, all our ghosts-tos and points-tos (in ConcLang and HeapLang) are fractional, but we have so far omitted this since it was not relevant for completeness. In λ_{Rust}, the completeness proof actually has to make use of fractions.

The solution is to choose $\mathsf{S}_\circ$ such that it tracks the accesses currently happening on all locations. If an access is ongoing, we may temporarily remove the points-to for the accessed location and replace it with a proof that no other thread can possibly be performing a concurrent access.[18] Ignoring concurrent reads for now, we can capture this intuition by requiring that if ℓ is a location in the heap with contents v, then:

$$\ell \mapsto_1 v \lor (\exists n, e.\, n \hookrightarrow^{\gamma}_{1/2} e * \ulcorner \textit{write-about-to-happen}(e) \urcorner)$$

That is, we either have the full points-to, or we know that some thread is about to perform a non-atomic write. To also handle non-atomic reads, the statement becomes more complicated and requires more detailed accounting of the fractional parts loaned out to each thread. In order to establish this formally, we use the generalized language-independent completeness theorem showcased in Appendix A, which exposes the threads' ghosts-to assertions to the client. We refer to our Rocq formalization for the full details.

Identified Completeness Gaps. While performing our completeness proofs, we noticed that the existing rules in λ_{Rust} were not complete. The missing rules were unrelated to data races, but instead about the semantics of comparisons, which can be non-deterministic when comparing pointers to already deallocated parts of memory. We presume that the absence of these rules was an oversight, since RustBelt never verified any programs that depended on this non-determinism.

Since λ_{Rust} uses a block-based memory model [46], the program logic also exposes *block tokens* tracking the size of each block, which we put into the state invariant $\mathsf{S}_\circ$. In doing so, we realized that these block tokens lacked an exclusivity law.

7 Case Study: Completeness of a Logic for Execution Time Bounds

Mével et al. [54] develop a logic for bounding the running time of programs by embedding time credits [4] into Iris. Here, we consider a completeness proof for such a logic. Similar to their approach, we extend ConcLang with a **tick** operation,[19] where $\mathbf{tick}(e)$ encodes the idea that executing e incurs a cost. For instance, if we want to bound the number of comparisons that a sorting function performs, we would replace each comparison $e_1 \le e_2$ in the function with $\mathbf{tick}(e_1 \le e_2)$. The grammar of this language is given in Appendix F.

For the semantics of **tick**, the state σ of the program is extended with a natural number counter field $\sigma.tc$. Evaluating $\mathbf{tick}(e)$ first evaluates e until it reaches a value v, then decrements the $\sigma.tc$ counter by one, and returns v. The **tick** operation gets stuck if the $\sigma.tc$ counter is zero. Thus, if $\sigma.tc = m$ and (e, σ) never gets stuck, then e performs no more than m **tick** operations.[20]

To reason about **tick**, the logic adds an assertion $\$(n)$ that represents permission to perform up to n **tick** operations. These assertions are called *time credits* and have the following rules:

TimecreditSplit	WpTick
$\$(m+n) \dashv\vdash \$(m) * \$(n)$	$\$(1) * \triangleright \Phi(v) \vdash \mathsf{wp}_\top\ \mathbf{tick}(v)\ \{\Phi\}$

TimecreditSplit allows us to split and join time credits with the separating conjunction. WpTick lets us reason about a **tick** operation by spending 1 credit.

To prove that a program does at most m **tick** operations, we prove a wp starting with an initial budget of $\$(m)$, as expressed in the logic's soundness theorem.

[18] This is inspired by a similar construction used to reason about optimizations that exploit the absence of data races [17, §3].
[19] Instead of extending ConcLang, Mével et al. [54] actually implement **tick** as a library in ConcLang. However, such an approach can only be complete for programs that use the library "correctly," which is hard to state, let alone reason about.
[20] It is essential to add **tick** around every operation we actually wish to count as incurring a cost. Mével et al. [54] discuss a translation that adds a **tick** to all steps of the language, while other work [26] has instead parameterized the operational semantics with a *cost model* for each transition, but the basic idea is the same.

Theorem 21 (Soundness of Time Credits)**.**
If $\$(m) \vdash \mathrm{wp}_{\top}\ e\ \{v.\ \ulcorner\varphi(v)\urcorner\}$*, then* $\mathit{safe}_{\varphi}(e,\sigma)$ *for all* σ *with* $\sigma.tc = m$*.*

Because this theorem implies e never gets stuck when starting from any state with $\sigma.tc = m$, we know that e performs at most m **tick** operations. Using the techniques of this paper, it is straightforward to prove a completeness result showing that the above two rules for time credits are all we need:

Theorem 22 (Completeness of Time Credits)**.**
If $\mathit{safe}_{\varphi}(e,\sigma)$ *for all* σ *with* $\sigma.tc = m$*, then* $\$(m) \vdash \mathrm{wp}_{\top}\ e\ \{v.\ \ulcorner\varphi(v)\urcorner\}$*.*

Proof Sketch. Apply the language-independent completeness theorem (Theorem 17), and instantiate $\mathrm{S}_{\circ}(\sigma)$ as $\left(\ast_{(\ell\leftarrow v)\in\sigma.h}\ \ell \mapsto v\right) * \$(\sigma.tc)$. The proof of Condition 18 is very similar to the one for ConcLang, and the only new case is for **tick**. For that case, from the safety assumption we know that $\sigma.tc$ is at least 1, so by using TimecreditSplit, we can pull out $\$(1)$, which we spend using WpTick to justify the **tick** step. □

8 Case Study: Completeness of a Logic for Probabilistic Error Bounds

In this section, we turn to Eris [1], an Iris-based separation logic for proving bounds on the probabilities of errors in higher-order sequential probabilistic programs. The idea is that many randomized programs are designed to satisfy a specification with very high probability, and only fail when some rare event occurs. Eris allows one to prove a bound on this probability of failure. Because Eris's specifications cover a probabilistic property, it does not use Iris's default definition of wp, so we cannot directly apply Theorem 17 or Theorem 20 to establish completeness. Nevertheless, as we will see, the general proof approach developed in the previous sections can be used to obtain a completeness result for Eris.

Eris targets a language that extends SeqLang with a probabilistic choice operator **rand** N, which evaluates to a random integer drawn uniformly from the set $\{0,\dots,N\}$. The grammar of this language is given in Appendix G. The language removes **free** and makes allocation deterministic, so that the only non-determinism is the probabilistic choice in **rand**. To define a semantics for this language, the authors first define the function $\mathrm{step} : \mathit{Expr}\times\mathit{State}\to\mathit{Expr}\times\mathit{State}\to[0,1]$, where $\mathrm{step}\ (e,\sigma)\ (e',\sigma')$ gives the probability that (e,σ) steps to (e',σ') in a single step. This plays an analogous role to the $\to$ relation in SeqLang. They then inductively define the function $\mathrm{exec}_n : \mathit{Expr}\times\mathit{State}\to\mathit{Val}\to[0,1]$, where $\mathrm{exec}_n\ (e,\sigma)\ v$ gives the probability that (e,σ) will terminate and return v in at most n steps. The partially applied function $\mathrm{exec}_n\ (e,\sigma)$ can be seen as a *sub-distribution* on values, meaning that the sum of the probabilities it assigns to all values is a number $p \le 1$. The gap between p and 1 represents the probability of non-termination after n steps. Finally, $\mathrm{exec}\ (e,\sigma)\ v$ is defined as the limit of exec_n as $n\to\infty$.

Eris comes in two variants, a *partial* version and a *total* version. In this section, we will focus on the total version, though our Rocq development proves a completeness result for the partial version as well. The key feature of Eris is the *error credit* assertion written $\lightning(\varepsilon)$ with $\varepsilon\in[0,1]$, which represents a "budget" or "permission" to err with probability ε. Just as time credits represent permission to incur a cost and must be spent every time a **tick** occurs, error credits are spent when a specification fails to hold with some probability. By constraining the initial budget, the probability of failure is bounded from above, as formalized in the soundness theorem for Total Eris:

Theorem 23 (Soundness of Total Eris)**.**
If $\lightning(\varepsilon) \vdash \mathrm{wp}\ e\ \left[v.\ \ulcorner\varphi(v)\urcorner\right]$*, then* $\Pr_{\mathrm{exec}\,(e,\sigma)}[\,\varphi\,] \ge 1-\varepsilon$ *for all* σ*.*

The conclusion of this statement says that with probability at least $1-\varepsilon$, evaluating (e,σ) will terminate with a value satisfying φ. In other words, the probability that it either diverges, gets stuck, or returns a value that fails to satisfy φ is at most ε, the initial error budget.

We reason about error credits using the following five rules:

ErrCombine
$$\lightning(\varepsilon_1) * \lightning(\varepsilon_2) \dashv\vdash \lightning(\varepsilon_1 + \varepsilon_2)$$

ErrWeaken
$$\lightning(\varepsilon_1) * \ulcorner \varepsilon_2 < \varepsilon_1 \urcorner \vdash \lightning(\varepsilon_2)$$

Err1
$$\lightning(1) \vdash \mathsf{False}$$

TwpRandExp
$$\frac{\sum_{i=0}^{N} \frac{\mathscr{E}(i)}{N+1} = \varepsilon}{\lightning(\varepsilon) \vdash \mathsf{wp}\ \mathbf{rand}\, N\ [n\ .\ \lightning(\mathscr{E}(n))]}$$

TwpThinAir
$$\frac{\forall \varepsilon'.\ \ulcorner \varepsilon' > \varepsilon \urcorner * \lightning(\varepsilon') * P \vdash \mathsf{wp}\ e\ [\Phi]}{\lightning(\varepsilon) * P \vdash \mathsf{wp}\ e\ [\Phi]}$$

ErrCombine allows us to split and join error credits, much like time credits. Using ErrWeaken, we can weaken our credits to a smaller amount. Err1 says that if we have 1 error credit, then we can derive False. The intuition here is that if we look at the statement of soundness, then owning $\lightning(1)$ means we only have to show our specification holds with probability at least $1-1=0$, which is trivial, so there is nothing to prove. The rule TwpRandExp allows us to re-distribute our credits across the different branches of $\mathbf{rand}\, N$, as long as the resulting error credits have the same *expected value* as the amount we started with. The user instantiates the rule with a function $\mathscr{E} : \{0,\dots,N\} \to [0,1]$, where $\mathscr{E}(i)$ represents how many error credits we will have in the postcondition when the random sample returns i. The idea is that, if an error will occur on some branch based on the outcome of $\mathbf{rand}\, N$, we shift credits from the non-erroring branches to the erroring branches, so that they have $\lightning(1)$ and can apply Err1. Finally, TwpThinAir[21] says that when proving a wp, if we start with ε credits, then it suffices to show that the proof can be carried out with any ε' that is strictly greater than ε. In other words, this rule allows us to conjure up additional error credits "out of thin air," but the additional error might be arbitrarily small. The justification for this rule relies on a continuity property of the wp, which we use to take the limit as ε' converges to ε.

Using these rules, it is possible to verify almost-sure termination of programs, *i.e.*, show that they terminate with probability 1. Almost-surely terminating programs are not necessarily strongly normalizing, but the probability of a diverging execution is 0. In general, reasoning about almost-sure termination is challenging, and the research literature has a number of variants of program logics and proof rules for showing almost-sure termination of probabilistic while loops, with recent interest in characterizing the completeness of these proof rules [49, 52].

We will show that Total Eris is *nearly* complete. The failure of completeness has nothing to do with probabilistic reasoning, but rather comes from the fact that the language uses a deterministic allocator, yet the proof rule for allocation is the same as the one we saw in §2, where we non-deterministically get a points-to for the returned location. Thus, instead we prove completeness for programs that do not allocate (but may make use of existing allocated locations):

Theorem 24 (Completeness of Total Eris)**.** *For all configurations (e,σ) not containing any memory allocation expressions, if $\Pr_{\mathrm{exec}\,(e,\sigma)}[\varphi] \geq 1-\varepsilon$, then $\lightning(\varepsilon) * \mathbin{\text{✻}}_{(\ell \leftarrow v)\in\sigma} \ell \mapsto v \vdash \mathsf{wp}\ e\ [v.\ \ulcorner \varphi(v) \urcorner]$.*

To approach the completeness theorem, we start from executions up to a finite number of steps.

Lemma 25. *For all configurations (e,σ) not containing any memory allocation expressions, if $\Pr_{\mathrm{exec}_n\,(e,\sigma)}[\varphi] \geq 1-\varepsilon$, then $\lightning(\varepsilon) * \mathbin{\text{✻}}_{(\ell \leftarrow v)\in\sigma} \ell \mapsto v \vdash \mathsf{wp}\ e\ [v.\ \ulcorner \varphi(v) \urcorner]$.*

Proof. We first weaken the $\lightning(\varepsilon)$ to $\lightning\left(1 - \Pr_{\mathrm{exec}_n\,(e,\sigma)}[\varphi]\right)$ via ErrWeaken, and then perform induction on n, with e and σ quantified. The structure mirrors the completeness proof in §2, but

[21] Note that the TwpThinAir rule was not a part of the original Eris logic [1], but only later introduced in the concurrent variant Coneris [47] and subsequently added to the Eris formalization as well.

with one key difference: whereas that proof performed induction on strong normalization (or, for partial programs, used Löb induction), here we induct on the index n of exec_n.

When $n = 0$, either $e = v$ for some v satisfying φ, or $\Pr_{\mathrm{exec}_0(e,\sigma)}[\varphi] = 0$. In both cases, the wp can be easily derived. In the inductive step, we need to show:

$$\begin{array}{l}\textit{If } \forall e', \sigma'.\ \lightning\left(1 - \Pr_{\mathrm{exec}_n(e',\sigma')}[\varphi]\right) * \mathbin{\text{✻}}_{(\ell \leftarrow v) \in \sigma'} \ell \mapsto v \vdash \mathsf{wp}\ e' \left[v. \ulcorner\varphi(v)\urcorner\right] \textit{ (the IH),} \\ \textit{then } \lightning\left(1 - \Pr_{\mathrm{exec}_{n+1}(e,\sigma)}[\varphi]\right) * \mathbin{\text{✻}}_{(\ell \leftarrow v) \in \sigma} \ell \mapsto v \vdash \mathsf{wp}\ e \left[v. \ulcorner\varphi(v)\urcorner\right].\end{array}$$

Here, the nontrivial case happens when e is a reducible expression (and hence not a value). As in §2, we decompose e into $K[e_1]$, where e_1 is base-reducible. We then apply WpBind and proceed by case distinction on the base step e_1 takes. For pure reductions and heap-dependent operations (only loads and stores are possible), the proof follows exactly the same pattern as before.

The genuinely new case arises when e_1 is $\mathbf{rand}\, N$. In that case, $e = K[\mathbf{rand}\, N]$ and the configuration (e, σ) steps to $(K[i], \sigma)$ for every $i = 0, \ldots, N$ with equal probability. The failure probability of this expression is given by:

$$1 - \Pr_{\mathrm{exec}_{n+1}(e,\sigma)}[\varphi] = \sum_{i=0}^{N} \frac{1 - \Pr_{\mathrm{exec}_n(K[i],\sigma)}[\varphi]}{N+1}$$

We can therefore apply TwpRandExp with $\mathscr{E}(i) \triangleq 1 - \Pr_{\mathrm{exec}_n(K[i],\sigma)}[\varphi]$. This yields $\lightning(\mathscr{E}(i))$ for each outcome i, with which we can obtain $\mathsf{wp}\ K[i] \left[v. \ulcorner\varphi(v)\urcorner\right]$ by the induction hypothesis. □

With this lemma, we can prove Theorem 24. We start by applying TwpThinAir, giving us $\lightning(\varepsilon')$ for some $\varepsilon' > \varepsilon$. From this inequality, we have that $\Pr_{\mathrm{exec}(e,\sigma)}[\varphi] > 1 - \varepsilon'$. By monotonicity of exec, there exists n such that $\Pr_{\mathrm{exec}_n(e,\sigma)}[\varphi] \geq 1 - \varepsilon'$. It follows by Lemma 25 that $\mathbin{\text{✻}}_{(\ell \leftarrow v) \in \sigma} \ell \mapsto v * \lightning(\varepsilon') \vdash \mathsf{wp}\ e \left[v. \ulcorner\varphi(v)\urcorner\right]$.

9 Case Study: Completeness of a Relational Logic

So far, we have looked at instances of Iris used for *unary* reasoning, where a specification describes a single program's behavior. However, Iris has also been used for *relational* reasoning, in which specifications relate the behavior of two programs. This kind of reasoning is useful because, among other applications, it can be used to show that a program *refines* a specification program. A common approach is to embed relational reasoning on top of an existing unary program logic by representing the specification program as a form of ghost state, as in the CaReSL logic [69]. With this approach, one introduces assertions to describe the state of this specification program. In this section, we will look at how this approach works for the case of the ConcLang language.

The logic comes with three new assertions. First, we have an assertion $j \Rightarrow e$ which states that the thread at index j in the spec program's thread pool is executing expression e, while $\ell \mapsto_s v$ says that in the spec program's heap, location ℓ points to v. Under the hood, both of these are just Iris ghost state, similar to $k \hookrightarrow^{\gamma} v$. Finally, we have an assertion $\mathrm{specCtx}^{\mathcal{N}}$ which represents the invariant that stores the authoritative copies of the spec program's state. This invariant ensures that the spec state can only be updated in a way that reflects the reduction semantics of the spec program. Then one derives rules for executing steps of the spec program by performing Iris ghost state updates. For example, for any $\mathcal{E}$ such that $\mathcal{N} \subseteq \mathcal{E}$, we have:

$$\begin{array}{l}\mathrm{specCtx}^{\mathcal{N}} * j \Rightarrow K[\ell \leftarrow w] * \ell \mapsto_s v \vdash \Rrightarrow_{\mathcal{E}} (j \Rightarrow K[()] * \ell \mapsto_s w) \\ \mathrm{specCtx}^{\mathcal{N}} * j \Rightarrow K[!\,\ell] * \ell \mapsto_s v \vdash \Rrightarrow_{\mathcal{E}} (j \Rightarrow K[v] * \ell \mapsto_s v) \\ \mathrm{specCtx}^{\mathcal{N}} * j \Rightarrow K[\mathbf{ref}(v)] \vdash \Rrightarrow_{\mathcal{E}} (\exists \ell.\ j \Rightarrow K[\ell] * \ell \mapsto_s v) \\ \mathrm{specCtx}^{\mathcal{N}} * j \Rightarrow K[\mathbf{fork}(e)] \vdash \Rrightarrow_{\mathcal{E}} (\exists j'.\ j \Rightarrow K[()] * j' \Rightarrow e)\end{array}$$

The first two rules perform stores and loads, respectively, using the spec program's points-to assertions for the corresponding location. The third rule allocates a new location, creating a new spec points-to for the new location. The fourth rule forks a thread and creates a fresh $j' \Rrightarrow e$ that represents the forked-off spec thread. Similar rules can be shown for all of the other primitive commands and pure steps of the language.

Then, to prove that an implementation e refines a specification e', one proves a particular wp for e, as captured by the following soundness theorem.

Theorem 26 (Relational Soundness). *If* $\mathsf{specCtx}^{\mathcal{N}} * 0 \Rrightarrow e' \vdash \mathsf{wp}\ e\ \{v.\ \exists v'.\ 0 \Rrightarrow v' * \ulcorner \varphi(v, v') \urcorner\}$, *then*

(1) $(e, \emptyset)$ *is safe, and*
(2) for all v, $\vec{e}$, *and* σ *such that* $([e], \emptyset) \to^*_{\mathtt{tp}} (v :: \vec{e}, \sigma)$ *there exists* v', $\vec{e}\,'$, *and* σ' *such that* $([e'], \emptyset) \to^*_{\mathtt{tp}} (v' :: \vec{e}\,', \sigma')$ *and* $\varphi(v, v')$.

This theorem requires a proof of the implementation e where in the precondition, specification ("spec") thread 0 executes e', and in the postcondition, the spec thread has reached some value v' that is related to the result v of the implementation. In the conclusion of the theorem, the first part states the usual safety of e that we expect from the wp.[22] The second part captures the relational nature of the logic: it shows that for every execution of e in which the first thread terminates in some value v, there is a corresponding execution of the spec program e' in which its first thread terminates in a value v' that is related to v under φ. In other words, this shows that e *refines* e'. Intuitively, this soundness theorem follows from the fact that, due to how specCtx and the spec points-to are defined, the only way we could have gotten $0 \Rrightarrow v'$ in the postcondition is by constructing such an execution of e'. Because the relational reasoning is constructed by an embedding on top of the unary wp, we get to reuse the soundness proof of the underlying wp. As a result, the proof of Theorem 26 is an easy consequence of the underlying soundness of the wp and the definition of the invariant in specCtx. We refer, *e.g.*, to Timany et al. [68] for the details.

Now, given that we know the underlying wp is also complete, can we also derive completeness of the relational logic, *i.e.*, a converse to Theorem 26? To show such a converse, we would get as assumptions that $(e, \emptyset)$ is safe and that e refines e' under relation φ, and we would need to derive the entailment. Applying the completeness property of the underlying wp (Theorem 16), we derive a wp about e of the following form:

$$\mathsf{wp}\ e\ \{v.\ulcorner \exists v', \vec{e}, \vec{e}\,', \sigma, \sigma'.\ ([e], \emptyset) \to^*_{\mathtt{tp}} (v :: \vec{e}, \sigma) \land ([e'], \emptyset) \to^*_{\mathtt{tp}} (v' :: \vec{e}\,', \sigma') \land \varphi(v, v')\urcorner\}$$

The postcondition here has the pure property about the existence of a corresponding execution of e'. To finish the proof, we now "just" need to show from this pure fact that it is possible to update $0 \Rrightarrow e'$ to $0 \Rrightarrow v'$. An obvious idea would be to do so by induction on $([e'], \emptyset) \to^*_{\mathtt{tp}} (v' :: \vec{e}\,', \sigma')$, showing that for each step in this trace, we can do an update on the ghost state.

However, perhaps surprisingly, this approach is blocked by one fundamental issue: the ghost update rule for allocation shown above. The conclusion of this allocation rule says that we just learn that there exists some location ℓ_0 at which the allocation has occurred. But because allocation is non-deterministic, ℓ_0 may not be the same location as the one where allocation happens in our assumed execution of e'!

In some sense, the notion of refinement we are working with is too brittle: we would like to say that the refinement does not depend on the specific locations selected by the allocation. In fact, by

[22] Unlike notions of refinement that are commonly used in compilation, where *any* e is said to refine e' if e' triggers undefined behavior, here the notion of refinement requires e to always be safe. We leave it to future work to see if our methods can prove completeness for relational logics that allow for "exploiting" undefined behavior.

changing the definition of the ghost state and the invariant in specCtx, we can derive the following *stronger* version of the soundness theorem:

Theorem 27 (Strong Relational Soundness)**.**
If $\mathsf{specCtx}^{\mathcal{N}} * 0 \Rrightarrow e' \vdash \mathsf{wp}\ e\ \{v.\ \exists v'.\ 0 \Rrightarrow v' * \ulcorner\varphi(v, v')\urcorner\}$*, then*

(1) $(e, \emptyset)$ *is safe, and*
(2) For any infinite set S *of locations, for all* $v, \vec{e}$*, and* σ *such that* $([e], \emptyset) \to^*_{\mathtt{tp}} (v :: \vec{e}, \sigma)$ *there exists* $v', \vec{e}\,', \sigma'$ *such that* $([e'], \emptyset) \to^*_{\mathtt{tp}} (v' :: \vec{e}\,', \sigma')$ *where* $\varphi(v, v')$ *and* $\mathsf{dom}(\sigma') \subseteq S$.

In other words, with this version of the theorem, we get to pick an infinite set S of locations,[23] and we get that for each execution of e there always exists a matching execution of e' *that only allocates locations in* S. This ensures that the executions of e' cannot depend too much on some particular choice of allocation. With this stronger definition of specCtx, we introduce an additional ghost state assertion $\mathsf{reserve}(S)$ that allows us to *reserve* a set of locations. This assertion is used in the following two rules for $\mathcal{N} \subseteq \mathcal{E}$:

$$\mathsf{specCtx}^{\mathcal{N}} \vdash \Rrightarrow_{\mathcal{E}} \exists S.\ \ulcorner\mathsf{infinite}(S)\urcorner * \mathsf{reserve}(S)$$

$$\mathsf{specCtx}^{\mathcal{N}} * j \Rrightarrow K[\mathtt{ref}(v)] * \mathsf{reserve}(S) * \ulcorner\ell \in S\urcorner \vdash \Rrightarrow_{\mathcal{E}} (j \Rrightarrow K[\ell] * \ell \mapsto_s v * \mathsf{reserve}(S \setminus \{\ell\}))$$

The first rule allows us to obtain, at any point, some infinite set S of reservations. The second rule says that if $\ell \in S$, where S is a set we have reserved, then we can force the allocation to be ℓ, which removes ℓ from the reservation set. With these stronger rules, we can prove the converse to Theorem 27 as a form of completeness.

Theorem 28 (Strong Relational Completeness)**.** *If* e *is an expression such that*

(1) $(e, \emptyset)$ *is safe, and*
(2) For any infinite set S *of locations, for all* $v, \vec{e}$*, and* σ *such that* $([e], \emptyset) \to^*_{\mathtt{tp}} (v :: \vec{e}, \sigma)$ *there exists* $v', \vec{e}\,', \sigma'$ *such that* $([e'], \emptyset) \to^*_{\mathtt{tp}} (v' :: \vec{e}\,', \sigma')$ *where* $\varphi(v, v')$ *and* $\mathsf{dom}(\sigma') \subseteq S$.

then $\mathsf{specCtx}^{\mathcal{N}} * 0 \Rrightarrow e' \vdash \mathsf{wp}\ e\ \{v.\ \exists v'.\ 0 \Rrightarrow v' * \ulcorner\varphi(v, v')\urcorner\}$.

The proof follows the sketch we started with above, except that we start by getting $\mathsf{reserve}(S)$ for some infinite set S. From our assumptions, we can then get that there exists an execution of e' that yields a related value where all allocations are in S. Now, because we have $\mathsf{reserve}(S)$, we can perform ghost updates that match this execution. By using the stronger ghost update rule for allocations, we ensure that all of the allocated locations in our ghost state match the ones in the execution we are given.

10 Semantic Conditions for Completeness

In earlier sections, we examined several existing instantiations of Iris and established the completeness of their proof rules. In this section, we turn to the question of what one has to consider, when coming up with a new instantiation of Iris, to ensure that a complete set of proof rules is even possible. For this purpose, we zoom out from any concrete case study and consider again the default Iris program logic with its language-independent wp. More precisely, we now look into the definition of wp, the ingredients that go into building an Iris program logic, and what conditions these ingredients need to uphold for completeness to become possible.

As mentioned in §3.3, this logic is first parameterized by a notion of a language Λ, which must satisfy some basic structural properties. The next parameter of the framework is the *state*

[23] More precisely, in our Rocq development, we use a form of constructive infinite sets that allows for splitting an infinite set into two infinite sets without using a choice principle.

interpretation predicate, $\mathsf{S}_\bullet$, which is a predicate over program states.[24] In Iris, assertions such as $\ell \mapsto v$ do not directly talk about the physical state; they are just ghost state. The role of $\mathsf{S}_\bullet$ is to define a relationship between that ghost state and the program's physical state. Under the hood, $\ell \mapsto v$ is defined using ghosts-tos, and the corresponding authoritative state is held by the state interpretation. This allows the user to define ghost state assertions that track the physical state. Since $\mathsf{S}_\bullet$ often needs to refer to some ghost state name γ, we sometimes write $\mathsf{S}_\bullet^\gamma$ to indicate this dependency on γ.

The $\mathsf{S}_\bullet$ is used by Iris in the *definition* of the wp. Whereas many program logics *define* the wp so that it is the weakest precondition, in Iris, the wp is instead defined recursively inside the logic.

Definition 29 (wp)**.** The $\mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\}$ assertion is defined as a guarded fixed point of the following recursive definition [36]

$$
\begin{aligned}
&(v \in \mathit{Val}) \quad \mathsf{wp}_{\mathcal{E}}\, v\, \{\Phi\} \triangleq \Rrightarrow_{\mathcal{E}} \Phi(v)\\
&(e \notin \mathit{Val}) \quad \mathsf{wp}_{\mathcal{E}}\, e\, \{\Phi\} \triangleq \forall \sigma.\, \mathsf{S}_\bullet(\sigma) \mathrel{-\!\!*} {}_{\mathcal{E}}\Rrightarrow_{\varnothing} \ulcorner \mathsf{red}(e,\sigma) \urcorner *\\
&\qquad\qquad \Big(\forall e', \sigma', \vec{e}_f.\, \ulcorner (e,\sigma) \rightarrow (e',\sigma',\vec{e}_f) \urcorner \mathrel{-\!\!*} \Rrightarrow_{\varnothing} \triangleright\, {}_{\varnothing}\Rrightarrow_{\mathcal{E}}\\
&\qquad\qquad \mathsf{S}_\bullet(\sigma') * \mathsf{wp}_{\mathcal{E}}\, e'\, \{\Phi\} * \mathop{\Large *}_{e_f \in \vec{e}_f} \mathsf{wp}_{\top}\, e_f\, \{_.\, \mathsf{True}\}\Big)
\end{aligned}
$$

The base case of this definition says that if e is already a value v, then the postcondition $\Phi(v)$ must hold immediately (under an update modality $\Rrightarrow_{\mathcal{E}}$). The inductive step consists of two parts. First, given the state interpretation for the current state σ, we must show $\mathsf{red}(e,\sigma)$, *i.e.*, that the current configuration is reducible. Then, for any e' that e may reduce to under σ, we must show the state interpretation holds for the new state σ', and that the wp holds for e' and any forked threads $\vec{e}_f$. This definition uses a mask-changing variant of the update modality, where the update can access some invariants and leave them open (or close some previously open invariants). The various update modalities along the way allow us to open invariants and modify ghost state in the course of proving these facts, while the later modality $\triangleright$ in the definition guards the recursive occurrence of the wp to ensure that the fixed point exists [11, 36]. The later modality here is also the reason why we can strip a later in hypotheses at each step.

On top of this definition, the Iris framework derives various language-independent rules about the wp, such as WpValue, WpWand, and WpAtomicInv. In turn, the user instantiating the framework is responsible for deriving wp rules for each of the language-specific primitives. These proofs require the use of dedicated *lifting lemmas* that relate wp to the operational semantics and the user's selected definition of $\mathsf{S}_\bullet$.

The Iris framework provides a generic soundness proof for this definition of wp. More precisely, in Iris this property is called *adequacy*, by analogy to the use of that term in denotational semantics, as it shows that the model of the wp actually says something about a program's behavior.

Theorem 30 (Language-Independent Adequacy)**.** *For all states σ, let $\mathsf{S}_\circ^\gamma$ be an Iris predicate on states such that*

$$\vdash \Rrightarrow \exists \gamma.\, \mathsf{S}_\bullet^\gamma(\sigma) * \mathsf{S}_\circ^\gamma(\sigma) \qquad \text{(StateAlloc)}$$

then for any meta-level predicate φ, $(\forall \gamma.\, \mathsf{S}_\circ^\gamma(\sigma) \vdash \mathsf{wp}_{\top}\, e\, \{v.\, \ulcorner \varphi(v) \urcorner\})$ implies $\mathit{safe}_\varphi(e,\sigma)$.

The condition of this theorem requires us to show the state interpretation and state invariant hold for some initial state σ. In turn, if we then derive a wp about e with $\mathsf{S}_\circ^\gamma(\sigma)$ as an assumption,

[24] This is somewhat simplified; we focus on what is relevant for this discussion.

we get that (e, σ) satisfies $safe_\varphi$. All of the specific soundness theorems we have looked at for partial correctness have been consequences of this language-independent adequacy theorem.

A natural question is whether we can show a kind of converse to this theorem under some assumptions about $\mathrm{S}_\bullet$ and $\mathrm{S}_\circ$. That is, can we show that whenever an expression satisfies *safe*, the wp holds? Such a converse would not exactly be a completeness theorem, because it would not tell us whether a particular collection of *proof rules* suffices for deriving the wp. But such a property is a necessary condition for completeness, since any complete logic would satisfy it. Thus, if our logic fails to satisfy this property, it has no hope of being complete. Moreover, establishing this would show that the wp is in fact the *weakest* precondition, *i.e.*, it is both a necessary and sufficient precondition. That is because any other sufficient precondition would imply *safe*, and therefore imply the wp.

We say that an instance of wp with this property is *requisite*. The following theorem gives sufficient conditions for requisiteness.

Theorem 31 (Language-Independent Requisiteness)**.** *For all γ, let $\mathrm{S}_\circ^\gamma$ be an Iris predicate on states such that*

$$\forall e, \sigma, \sigma_l.\ \ulcorner \mathsf{red}(e, \sigma_l) \urcorner \mathrel{-\!\!*} \mathrm{S}_\bullet^\gamma(\sigma) \mathrel{-\!\!*} \mathrm{S}_\circ^\gamma(\sigma_l) \mathrel{-\!\!*} \Rrightarrow_\top \ulcorner \mathsf{red}(e, \sigma) \urcorner * \tag{StateUpd}$$
$$\left(\forall e', \sigma', \vec{e}_f.\ \ulcorner (e, \sigma) \rightarrow (e', \sigma', \vec{e}_f) \urcorner \mathrel{-\!\!*} \Rrightarrow_\top \exists \sigma_l'.\ \ulcorner (e, \sigma_l) \rightarrow (e', \sigma_l', \vec{e}_f) \urcorner * \mathrm{S}_\bullet^\gamma(\sigma') * \mathrm{S}_\circ^\gamma(\sigma_l')\right)$$

then for any meta-level predicate φ, $safe_\varphi(e, \sigma)$ implies $\mathrm{S}_\circ^\gamma(\sigma) \vdash \mathsf{wp}_\top\, e\, \{v.\ \ulcorner \varphi(v) \urcorner\}$.

To get some intuition for the condition StateUpd, we think of $\mathrm{S}_\bullet$ as the "authoritative" full description of the physical state σ, whereas $\mathrm{S}_\circ$ describes some "fragment" or subset of the state σ_l. Then, if e is reducible in the subset of state σ_l for which we have $\mathrm{S}_\circ$, and we are given $\mathrm{S}_\bullet$ for the full state σ, then we must first be able to show that e is also reducible in σ. Furthermore, for every e' that e can step to from σ, we must update the $\mathrm{S}_\bullet$ accordingly, and moreover, we must show that there is a corresponding step to e' from σ_l that forks the same threads. For this step from σ_l, we have to be able to update $\mathrm{S}_\circ$.

This captures a sort of locality of the program semantics: ownership of σ_l is enough to characterize how e steps in any larger state that is compatible with ownership of $\mathrm{S}_\circ^\gamma(\sigma_l)$.

Proof sketch of Theorem 31. As in the proof of Theorem 16, we first allocate ghost state that will be used to track the status of all concurrent threads that are created. Exactly like before, we create an invariant that stores $\mathrm{S}_\circ$, the ghost map for the threads, and the pure fact that the configuration satisfies $safe\text{-}tp_\varphi$.

$$I_{compl} \triangleq \boxed{\exists \vec{e}, \sigma_l.\ \ulcorner safe\text{-}tp_\varphi(\vec{e}, \sigma_l) \urcorner * \bullet^\gamma \vec{e} * \mathrm{S}_\circ(\sigma_l)}^{\mathcal{N}}$$

Still using Löb induction, we prove $I_{compl} * n \hookrightarrow^\gamma e \vdash \mathsf{wp}_\top\, e\, \{v.\ n \hookrightarrow^\gamma v\}$, which has the same shape as the entailment in Lemma 14. However, rather than inverting on the base step as we did in that lemma (which we cannot do here as this proof is generic with respect to the language Λ), we instead unfold the definition of the wp. The proof is trivial if e is a value. Otherwise, we open the invariant to show that e can execute for one step and update the thread pool accordingly. We can do these updates by applying StateUpd. We then close the invariant with the updated state and thread pool, and use the induction hypothesis to conclude. □

Discussion: Soundness, Completeness, Adequacy, and Requisiteness. Although Theorem 17 and Theorem 31 reach conclusions of the same form, the two theorems carry very different meanings. Completeness of a program logic, as the converse of soundness, is a property of a set of reasoning rules, and can hold even without having any semantic model of the assertions.

Requisiteness, as the converse of adequacy, is a property of a semantic model, and can hold in the absence of any reasoning rules. For example, a syntactic program logic with an opaque wp and axiomatized reasoning rules can be complete as long as those axioms satisfy Condition 18, regardless of whether the wp has a semantic interpretation. Symmetrically, a wp predicate alone can be requisite even if the logic provides no reasoning rules. In this sense, completeness abstractly characterizes the external interface of a program logic, whereas requisiteness concerns a possible internal implementation of that logic. The two notions are of course connected: for a semantics-based program logic, completeness implies requisiteness, dually to the way adequacy of the model implies soundness of the logic.

Particularly for Iris, these two theorems help us identify the sources of incompleteness when designing a program logic. If StateUpd holds but Condition 18 fails, then one has designed a well-behaved operational semantics and a sufficiently complete state interpretation, but probably "forgotten" some rules in the program logic (the primitive laws) or these laws otherwise fail to capture the full power of the state interpretation. The λ_{Rust} case study in §6 finds several such forgotten rules. On the other hand, if StateUpd also fails, then this suggests that the operational semantics is problematic or the underlying state interpretation is incomplete for certain programs, which should be addressed first. The problem with **`resolve`** in HeapLang is of this nature since the operational semantics was erroneous, as described in §4.

11 Related Work

Cook's Original Completeness Result. Cook [12] gave the first form of a completeness proof for a variant of Hoare logic for an Algol-like language with non-recursive procedure calls and while loops. He noted that if one considers a fully syntactic proof system for both the assertion logic and the Hoare triple rules, then when the programming language under consideration is sufficiently expressive, any such proof system must be incomplete, since otherwise it could be used to decide the halting problem. His key insight was that one can side-step this problem and focus on whether there is any incompleteness that can be attributed to a deficiency in the Hoare logic rules, as opposed to the assertion logic rules. To do so, he examines completeness of the Hoare triple rules under the assumption that one has a complete proof system for the assertion logic, a property that has come to be called *relative* completeness. In the case of Iris, we effectively do something similar, since Iris embeds pure assertions from the meta-logic, and adds a proof rule that reflects all implications from the meta-logic as entailments between pure assertions.

Cook first establishes relative completeness under the assumption that for any expression e and precondition P, there exists a (semantically) strongest postcondition assertion $\mathrm{sp}(e, P)$ expressible in the logic. He then shows that $\vdash \{P\}\ e\ \{\mathrm{sp}(e, P)\}$ is syntactically derivable. To do so, he first considers a version e' of the program obtained by inlining all procedure calls, and proceeds by induction on the length of e' plus the number of inlining steps needed to derive e' from e. The main challenge lies in the case for while loops, where one has to construct a loop invariant. Once that is done, relative completeness follows by observing that if $\{P\}\ e\ \{Q\}$ is semantically valid, then $\mathrm{sp}(e, P) \Rrightarrow Q$ by construction. Hence, $\mathrm{sp}(e, P) \vdash Q$ by the assumed completeness of the assertion logic, and so from $\vdash \{P\}\ e\ \{\mathrm{sp}(e, P)\}$, we can derive $\vdash \{P\}\ e\ \{Q\}$ by the rule of consequence. Finally, he shows that for a sufficiently expressive assertion logic, $\mathrm{sp}(e, P)$ can indeed be defined syntactically.

In contrast, for our proofs, rather than working with a semantically defined weakest precondition or strongest postcondition, we effectively end up showing that the wp *provided* by Iris is in fact the *weakest* precondition. Because we consider a language with general higher-order functions and state, we cannot inline procedure calls and proceed by induction on the length of the program as Cook does. Instead, we used Löb induction and did case analysis on the reduction steps of e. This use of Löb induction as the primitive for reasoning about recursion also means that we side-stepped

the challenge Cook faced in deriving a loop invariant for the while rule. Our key challenge was to find a suitable Löb induction invariant where Cook had to find a suitable loop invariant.

Completeness for Concurrency Logics. Owicki and Gries [59] introduced a proof system for reasoning about concurrent programs. With their approach, to apply the parallel composition rule to $e_1 \parallel e_2$, we need not just Hoare *triples* about e_1 and e_2, but rather full Hoare proof outlines that include each intermediate assertion in the Hoare proof between program statements. Then one checks a non-interference property that requires showing each intermediate assertion used in e_1 is *stable* under each of the possible steps of e_2 and vice versa. Owicki [58] proved the relative completeness of a restricted version of this system, which uses monitor-like resources [29] as the synchronization primitive, and only allows for a single parallel composition of n statements. The idea behind her proof is to start by annotating a program with a collection of *auxiliary variables*. These variables are defined so that by inspecting their values during execution, one can determine the relevant history of the program's execution and interleaving order up to that point. Using these variables, she defines intermediate assertions that are stable by construction, because they can characterize the reachable states of the program. She suggests that the proof of completeness can be generalized to a language with richer synchronization primitives and nested parallelism, but notes that this would complicate the proof. In contrast, our technique straightforwardly scales to much richer languages with fine-grained concurrency and dynamic threads. This is because we use Iris's more general support for ghost resources in order to track the state of each thread, instead of annotating the program with auxiliary variables that encode execution ordering. In particular, the ghosts-to assertions are defined in a way that does not depend on the specific details of the language's primitives for concurrency and synchronization.

One drawback of the Owicki-Gries method is that it is non-modular, because the parallel composition rule requires re-inspecting the proofs of the threads and checking them against each other. Rely-guarantee [34] addresses this by using predicates to abstract over the behaviors of threads: we verify each thread under a *rely* assertion that describes what interference could be caused by its environment, while showing that the thread's behavior upholds a *guarantee* assertion. Then, for parallel composition, we simply check for the compatibility of each thread's rely assertion against the other thread's guarantee. Stirling [64] observes that rely-guarantee is complete relative to Owicki-Gries, and since the latter is complete, so too is rely-guarantee. Xu et al. [71] give a direct proof of relative completeness for rely-guarantee, adapting and simplifying the auxiliary variable construction of Owicki. In Iris, rely-guarantee style reasoning is subsumed by the use of invariants and appropriate ghost state.

Most recently, de Boer and Hiep [14] establish relative completeness of a concurrent separation logic by adapting Owicki's technique for proving relative completeness. They define an abstract generalized version of the idea behind Owicki's proof, which can then be instantiated to derive completeness of different logics. They apply this in particular to a CSL that supports dynamic thread creation. As compared to our work, they restrict attention to a first-order language, whereas we have considered higher-order languages with higher-order state. On the other hand, their general theorem would apply to any logic that satisfies their abstract conditions, whereas we have focused on logics built on the Iris framework and its logical mechanisms.

Sequential Separation Logics. Ishtiaq and O'Hearn [33] establish the completeness of a sequential separation logic that uses Hoare triples by showing that the weakest precondition can be formulated as an assertion in the logic. Yang [72] and Yang and O'Hearn [73] also establish completeness results for the original formulation of sequential separation logic.

Haslbeck and Nipkow [27] prove the completeness of three program logics for reasoning about time bounds, including a separation logic with time credits. They restrict attention to a language

without a dynamic heap or allocation, so as to be able to compare to the other two non-separation logics under consideration. Our results in §7 consider time credits in a language with allocation.

All of the prior completeness results mentioned so far restrict attention to first-order programs. Honda et al. [30] developed a complete program logic for imperative higher-order programs using so-called characteristic formulae that completely capture the behavior of a program. They note that in the first-order case, such formulae are easily constructed, but that in the higher-order case, the challenge is that the description of a higher-order function's behavior must be somehow parameterized by the behavior of its input. Charguéraud [10] adapted characteristic formulae to the setting of higher-order separation logic and established relative completeness in that setting. In our approach, we side-step the question of finding a way to characterize a program's behavior as a formula in the logic by instead representing the threads of the program as ghost state and embedding pure assertions about safety in our invariants.

12 Conclusion

We have introduced a general pattern for completeness proofs of Iris-based program logics. For the family of logics that are based on Iris's shared program logic, we provide reusable lemmas that simplify such proofs for future work. As part of this, we have identified several small gaps in existing logics, indicating that indeed these proofs do the job that one would expect from completeness: ensuring that the proof rules cover everything relevant about every construct in the language.

Our work also sheds some light on which features of Iris are "necessary" for completeness, namely Löb induction, fancy updates with first-order invariants, and ghost maps (with their ghosts-tos). Löb induction is foundational to our work, since it allows us to handle complicated programs that, *e.g.*, load higher-order functions from the heap, without having to find syntactic pre- or postconditions for such indirect calls. Invariants and ghost maps can be seen as a separation logic form of rely-guarantee reasoning or auxiliary variables; they show up in much the same way that, *e.g.*, auxiliary variables are used in Owicki's completeness proof (see §11). Our proof also relies on the fact that WpValue and WpBind are bi-entailments, even though the backwards direction is rarely needed in Iris proofs.

Interestingly, most of the features that make Iris so expressive are not needed for completeness. We did not need additional higher-order ghost state, or even any form of user-defined ghost state besides ghost maps, and while we did use invariants, all our invariants are timeless, meaning we did not have a need for general impredicative invariants. Arguably, this is not very surprising. Features like impredicative invariants and higher-order / user-defined ghost state are motivated by the desire to give powerful, modular specifications to interesting *libraries*, while completeness talks about proving correctness of a *whole program* in a single, monolithic proof. Library case studies and proofs of completeness are orthogonal means of assessing the expressivity of a logic. Iris has been extensively evaluated for the former; by adding the latter, our work closes a long-standing gap in the work on Iris.

Data Availability Statement

The Rocq mechanization accompanying this work is available online on Zenodo [31]. The latest version can be found on the MPI-SWS GitLab at https://gitlab.mpi-sws.org/johannes/iris-completeness.

Acknowledgments

This work was supported in part by the National Science Foundation under Grant No. 2338317 and Grant No. 2319168, as well as the Carlsberg Foundation under Grant No. CF23-0791. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of these funding agencies.

A More General Language-Independent Completeness

The more general form of Theorem 17 that we used in §6 is obtained by replacing Condition 18 with the following:

Condition 32 (Completeness)**.**

$$\ulcorner\text{base-red}(e,\sigma)\urcorner * \mathrm{S}_\circ(\sigma) * \bullet^\gamma \vec{e} * \ulcorner \mathit{safe\text{-}tp}_\varphi(\vec{e},\sigma)\urcorner * n \hookrightarrow^\gamma K[e] \vdash \Rrightarrow_{\mathcal{E}}$$
$$\left(\ulcorner\text{Atomic } e\urcorner * \forall\Phi.\ \triangleright\ \mathit{WithStatePre}(\mathcal{E},e,n,K,\sigma,\vec{e},\Phi) \mathrel{-\!\!*} \mathrm{wp}_{\mathcal{E}}\, e\, \{\Phi\}\right) \vee$$
$$\left(\mathrm{S}_\circ(\sigma) * \bullet^\gamma \vec{e} * \forall\Phi.\ \triangleright\ \mathit{WithoutStatePre}(\mathcal{E},e,K,\Phi) \mathrel{-\!\!*} \mathrm{wp}_\top\, e\, \{\Phi\}\right)$$

where

$$\mathit{WithStatePre}(\mathcal{E},e,n,K,\sigma,\vec{e},\Phi) \triangleq \forall v',\sigma',\vec{e}_f.\ \ulcorner(e,\sigma) \to (v',\sigma',\vec{e}_f)\urcorner * \mathrm{S}_\circ(\sigma') * \bullet^\gamma \vec{e} * n \hookrightarrow^\gamma K[e] \mathrel{-\!\!*}$$
$$\Rrightarrow_{\mathcal{E}}\ \Phi(v') * \textstyle\ast_{e_f\in\vec{e}_f} \mathrm{wp}_\top\, e_f\, \{v.\,\mathrm{True}\}$$
$$\mathit{WithoutStatePre}(\mathcal{E},e,K,\Phi) \triangleq \forall e',\vec{e},\vec{e}_f.\ \mathit{Reduces}(e,K,e',\vec{e}) \mathrel{-\!\!*} \Rrightarrow_\top$$
$$\mathrm{wp}_\top\, e'\, \{\Phi\} * \textstyle\ast_{e_f\in\vec{e}_f} \mathrm{wp}_\top\, e_f\, \{v.\,\mathrm{True}\}$$
$$\mathit{Reduces}(e,K,e',\vec{e}) \triangleq \forall\sigma.\ \mathrm{S}_\circ(\sigma) * \bullet^\gamma \vec{e} * \ulcorner \mathit{safe\text{-}tp}_\varphi(\vec{e},\sigma)\urcorner \mathrel{-\!\!*} \Rrightarrow_{\mathcal{E}}$$
$$\exists\sigma'.\ \ulcorner(e,\sigma) \to^+ (e',\sigma',\vec{e}_f)\urcorner * \mathrm{S}_\circ(\sigma') * \bullet^\gamma \vec{e} * n \hookrightarrow^\gamma K[e]$$

The key difference from Theorem 17 is that this version also exposes ownership of the thread pool and per-thread ghosts-to assertions. These resources have to be given back unchanged, but making them available to the proof temporarily is crucial for §6.

B Stronger Sequential Versions of Completeness and Requisiteness Theorems

The wp derived by Theorem 17 takes the state invariant $\mathrm{S}_\circ(\sigma)$ in the precondition but only gives back a meta-level postcondition $\varphi(v)$. It turns out that if we restrict the expression e to be sequential, we can derive a stronger version of completeness that gives back a $\mathrm{S}_\circ$ for the final state.

To keep the result language-agnostic, we define sequentiality as a meta-level semantic property saying that the thread pool only ever contains 1 thread:

$$\mathit{seq}(e,\sigma) \triangleq \forall\,\vec{e}\,',\sigma'.\ ([e],\sigma) \to^*_{\mathtt{tp}} (\vec{e}\,',\sigma') \Rightarrow |\,\vec{e}\,'\,| = 1$$

We also extend the notion of *safe* to account for the final state.

$$\mathit{ssafe\text{-}tp}_\phi(\vec{e},\sigma) \triangleq \forall\,\vec{e}\,',\sigma'.\ (\vec{e},\sigma) \to^*_{\mathtt{tp}} (\vec{e}\,',\sigma') \implies$$
$$\forall n, e''.\ \vec{e}\,'(n) = e'' \Rightarrow (\exists v.\ e'' = v \wedge (n = 0 \Rightarrow \phi(v,\sigma'))) \vee \mathrm{red}(e'',\sigma')$$
$$\mathit{ssafe}_\phi(e,\sigma) \triangleq \mathit{ssafe\text{-}tp}_\phi([e],\sigma)$$

The sequential version of completeness and requisiteness is then

Theorem 33 (Language-Independent Sequential Completeness)**.** *Let Λ be a language satisfying Condition 32. If $\mathit{ssafe}_\phi(e,\sigma) \wedge \mathit{seq}(e,\sigma)$, then $\mathrm{S}_\circ(\sigma) \vdash \mathrm{wp}_\top\, e\, \{v.\,\exists\sigma'.\ \mathrm{S}_\circ(\sigma') * \ulcorner\phi(v,\sigma')\urcorner\}$.*

Theorem 34 (Language-Independent Sequential Requisiteness)**.** *For all γ, let $\mathrm{S}^\gamma_\circ$ be an Iris predicate on states satisfying StateUpd, then for any meta-level predicate ϕ, $\mathit{ssafe}_\phi(e,\sigma) \wedge \mathit{seq}(e,\sigma)$ implies $\mathrm{S}^\gamma_\circ(\sigma) \vdash \mathrm{wp}_\top\, e\, \{v.\,\exists\sigma'.\ \mathrm{S}^\gamma_\circ(\sigma') * \ulcorner\phi(v,\sigma')\urcorner\}$.*

Sequentiality here is used in an essential way. When the main thread terminates, it must frame a fragment of the final state, σ', into $\mathrm{S}_\circ(\sigma')$ to conclude its postcondition. This prevents other threads from accessing σ' from this point on. If we allow e to fork, then we would have to show the child threads can still make progress after the main thread terminates, without accessing σ'.

C The Full Rules of the Iris Logic

Most of this section is quoted from [32]. Iris embeds all the usual connectives from higher-order logic. The quantifiers have the usual rules of variable freshness, which we omit here. Quantifiers are impredicative in that they can quantify over Iris propositions, as well as regular Rocq types; with the judgment $t : \tau$ enforcing well-typedness.

$$P \vdash P \qquad \frac{P \vdash Q \qquad Q \vdash R}{P \vdash R} \qquad P \vdash \mathsf{True} \qquad \frac{P \vdash Q \qquad P \vdash R}{P \vdash Q \land R} \qquad \frac{P \vdash Q \land R}{P \vdash Q} \qquad \frac{P \vdash Q \land R}{P \vdash R}$$

$$\frac{P \vdash Q}{P \vdash Q \lor R} \qquad \frac{P \vdash R}{P \vdash Q \lor R} \qquad \frac{P \vdash R \qquad Q \vdash R}{P \lor Q \vdash R} \qquad \frac{P \land Q \vdash R}{P \vdash Q \Rightarrow R} \qquad \frac{P \vdash Q \Rightarrow R \qquad P \vdash Q}{P \vdash R}$$

$$\frac{P \vdash Q}{P \vdash \forall x : \tau.\, Q} \qquad \frac{P \vdash \forall x : \tau.\, Q \qquad t : \tau}{P \vdash Q[t/x]} \qquad \frac{P \vdash Q[t/x] \qquad t : \tau}{P \vdash \exists x : \tau.\, Q} \qquad \frac{P \vdash Q}{\exists x : \tau.\, P \vdash Q}$$

The laws for separating conjunction, the magic wand, and the pure embedding $\ulcorner \varphi \urcorner$ are shown here. Note that the laws for the pure embedding are not purely syntactic but instead manipulate the meta-level context.

$$\begin{array}{c} \mathsf{True} * P \dashv\vdash P \\ P * Q \vdash Q * P \\ (P * Q) * R \vdash P * (Q * R) \end{array} \qquad \frac{P_1 \vdash Q_1 \qquad P_2 \vdash Q_2}{P_1 * P_2 \vdash Q_1 * Q_2} \qquad \frac{P * Q \vdash R}{P \vdash Q \mathrel{-\!\!*} R}$$

$$\frac{\varphi \text{ holds in the meta-logic}}{\vdash \ulcorner \varphi \urcorner} \qquad \frac{P \vdash \ulcorner \varphi \urcorner \qquad \varphi \implies (P \vdash Q)}{P \vdash Q}$$

The laws for the later modality include the Löb law. We also give the laws for the persistence modality $\Box P$, which is used internally in the infrastructure that we use to do our proofs.

$$\frac{P \vdash Q}{\triangleright P \vdash \triangleright Q} \qquad P \vdash \triangleright P \qquad \frac{\triangleright P \vdash P}{\vdash P} \qquad \begin{array}{c} \forall x.\, \triangleright P \vdash \triangleright \forall x.\, P \\ \triangleright \exists x.\, P \vdash \triangleright \mathsf{False} \lor \exists x.\, \triangleright P \\ \triangleright P \vdash \triangleright \mathsf{False} \lor (\triangleright \mathsf{False} \Rightarrow P) \\ \triangleright (P * Q) \dashv\vdash \triangleright P * \triangleright Q \end{array}$$

$$\frac{P \vdash Q}{\Box P \vdash \Box Q} \qquad \Box P \vdash P \qquad \begin{array}{c} \Box P \land Q \vdash \Box P * Q \\ \Box \triangleright P \dashv\vdash \triangleright \Box P \end{array} \qquad \begin{array}{c} \Box P \vdash \Box \Box P \\ \forall x.\, \Box P \vdash \Box \forall x.\, P \\ \Box \exists x.\, P \vdash \exists x.\, \Box P \end{array}$$

Finally, we have the laws for the fancy update ${}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2}$. Note that most of the paper does not need the mask-changing version.

$$\frac{P \vdash Q}{{}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2} P \vdash {}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2} Q} \qquad \frac{\mathcal{E}_2 \subseteq \mathcal{E}_1}{P \vdash {}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2}\, {}_{\mathcal{E}_2}\!\Rightarrow\!{}_{\mathcal{E}_1} P} \qquad {}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2}\, {}_{\mathcal{E}_2}\!\Rightarrow\!{}_{\mathcal{E}_3} P \vdash {}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_3} P \qquad P \vdash \Rightarrow_{\mathcal{E}} P$$

$$Q * {}_{\mathcal{E}_1}\!\Rightarrow\!{}_{\mathcal{E}_2} P \vdash {}_{\mathcal{E}_1 \uplus \mathcal{E}_f}\!\Rightarrow\!{}_{\mathcal{E}_2 \uplus \mathcal{E}_f} Q * P \qquad \frac{\mathsf{timeless}(P)}{\triangleright P \vdash \Rightarrow_{\mathcal{E}} P}$$

We do not include the laws for the plainly modality $\blacksquare P$ since we do not use this modality at all. Similarly, we omit the laws for $\mathsf{Own}\,(a)$ since we do not need custom ghost state. We only need the ghost maps introduced in §3.1.

D The Full Definition of the HeapLang Logic

Most of this section is quoted from [32]. Changes to address incompletenesses identified in this work are marked as blue.

Syntax.

$$
\begin{aligned}
v, w \in \mathit{Val} &::= z \mid b \mid () \mid \lightning \mid \ell \mid p && (z \in \mathbb{Z}, b \in \mathbb{B}, \ell \in \mathit{Loc}, p \in \mathit{ProphId})\\
&\mid \mathbf{rec}\ f(x) = e \mid (v, w) \mid \mathbf{inl}(v) \mid \mathbf{inr}(v) && (\circledcirc_1 \in \{-, \ldots\})\\
e \in \mathit{Expr} &::= v \mid x \mid \mathbf{rec}\ f(x) = e \mid e_1(e_2) \mid \circledcirc_1 e \mid e_1 \circledcirc_2 e_2 && (\circledcirc_2 \in \{+, -, +_{\mathsf{L}}, =, \ldots\})\\
&\mid \mathbf{if}\ e\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \mid (e_1, e_2) \mid \mathbf{fst}(e) \mid \mathbf{snd}(e)\\
&\mid \mathbf{inl}(e) \mid \mathbf{inr}(e) \mid (\mathbf{match}\ e\ \mathbf{with}\ \mathbf{inl} \Rightarrow e_1 \mid \mathbf{inr} \Rightarrow e_2\ \mathbf{end}) \mid \mathbf{AllocN}(e_1, e_2)\\
&\mid \mathbf{free}(e) \mid !\, e \mid e_1 \leftarrow e_2 \mid \mathbf{CmpXchg}(e_0, e_1, e_2) \mid \mathbf{Xchg}(e_1, e_2) \mid \mathbf{FAA}(e_1, e_2) \mid \mathbf{fork}(e)\\
&\mid \mathbf{newproph} \mid \mathbf{resolve}\ \mathbf{with}\ e_1\ \mathbf{at}\ e_2\ \mathbf{to}\ e_3\\
K \in \mathit{Ctx} &::= \bullet \mid K_>\\
K_> \in \mathit{Ctx}_> &::= e(K) \mid K(v) \mid \circledcirc_1 K \mid e \circledcirc_2 K \mid K \circledcirc_2 v \mid \mathbf{if}\ K\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \mid (e, K) \mid (K, v)\\
&\mid \mathbf{fst}(K) \mid \mathbf{snd}(K) \mid \mathbf{inl}(K) \mid \mathbf{inr}(K) \mid (\mathbf{match}\ K\ \mathbf{with}\ \mathbf{inl} \Rightarrow e_1 \mid \mathbf{inr} \Rightarrow e_2\ \mathbf{end})\\
&\mid \mathbf{AllocN}(e, K) \mid \mathbf{AllocN}(K, v) \mid \mathbf{free}(K) \mid !\, K \mid e \leftarrow K \mid K \leftarrow v\\
&\mid \mathbf{CmpXchg}(e_0, e_1, K) \mid \mathbf{CmpXchg}(e_0, K, v_2) \mid \mathbf{CmpXchg}(K, v_1, v_2)\\
&\mid \mathbf{Xchg}(e, K) \mid \mathbf{Xchg}(K, v) \mid \mathbf{FAA}(e, K) \mid \mathbf{FAA}(K, v)\\
&\mid \mathbf{resolve}\ \mathbf{with}\ e_1\ \mathbf{at}\ e_2\ \mathbf{to}\ K \mid \mathbf{resolve}\ \mathbf{with}\ e_1\ \mathbf{at}\ K\ \mathbf{to}\ v_3\\
&\mid \mathbf{resolve}\ \mathbf{with}\ K_>\ \mathbf{at}\ v_2\ \mathbf{to}\ v_3
\end{aligned}
$$

$$\mathbf{ref}(e) \triangleq \mathbf{AllocN}(1, e) \qquad \mathbf{CAS}(e_0, e_1, e_2) \triangleq \mathbf{snd}(\mathbf{CmpXchg}(e_0, e_1, e_2))$$

Semantics. The state of HeapLang is a record of a heap and a set of used prophecy IDs.

$$\ell \in \mathit{Loc} \triangleq \mathbb{Z} \qquad p \in \mathit{ProphId} \triangleq \mathbb{Z}^+ \qquad \sigma \in \mathit{State} \triangleq \{h \colon \mathit{Loc} \xrightarrow{\text{fin}} \mathit{Val}^?;\ \mathit{pid} \colon \wp(\mathit{ProphId})\}$$

$$\rho \in \mathit{Conf} \triangleq \mathcal{L}(\mathit{Expr}) \times \mathit{State} \qquad \kappa \in \mathit{Obs} \triangleq \mathit{ProphId} \times (\mathit{Val} \times \mathit{Val})$$

Pure Reductions.

$$(\mathbf{rec}\ f(x) = e)(v) \to_{\mathbf{pure}} e[(\mathbf{rec}\ f(x) = e)/f][v/x] \qquad -_\circledcirc\, z \to_{\mathbf{pure}} -z \qquad z_1 +_\circledcirc z_2 \to_{\mathbf{pure}} z_1 + z_2$$

$$z_1 -_\circledcirc z_2 \to_{\mathbf{pure}} z_1 - z_2 \qquad \ell +_{\mathsf{L}} z \to_{\mathbf{pure}} \ell + z \qquad \frac{v_1 \cong v_2}{v_1 =_\circledcirc v_2 \to_{\mathbf{pure}} \mathbf{true}} \qquad \frac{v_1 \ncong v_2}{v_1 =_\circledcirc v_2 \to_{\mathbf{pure}} \mathbf{false}}$$

$$\mathbf{if}\ \mathbf{true}\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \to_{\mathbf{pure}} e_1 \qquad \mathbf{if}\ \mathbf{false}\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \to_{\mathbf{pure}} e_2$$

$$\mathbf{fst}((v_1, v_2)) \to_{\mathbf{pure}} v_1 \qquad \mathbf{match}\ \mathbf{inl}(v)\ \mathbf{with}\ \mathbf{inl} \Rightarrow e_1 \mid \mathbf{inr} \Rightarrow e_2\ \mathbf{end} \to_{\mathbf{pure}} e_1(v)$$

$$\mathbf{snd}((v_1, v_2)) \to_{\mathbf{pure}} v_2 \qquad \mathbf{match}\ \mathbf{inr}(v)\ \mathbf{with}\ \mathbf{inl} \Rightarrow e_1 \mid \mathbf{inr} \Rightarrow e_2\ \mathbf{end} \to_{\mathbf{pure}} e_2(v)$$

Impure Base Reductions.

$$\frac{z > 0 \qquad \forall i \in [0, z).\ \ell + i \notin \mathrm{dom}(\sigma.h)}{(\mathbf{AllocN}(z, v), \sigma) \to_{\mathbf{base}} (\ell,\ \sigma \text{ with } h := \sigma.h[[\ell, \ell + z) \leftarrow v]\,, \epsilon)}$$

$$\frac{\sigma.h(\ell) \neq \bot}{(\mathbf{free}(\ell), \sigma) \to_{\mathbf{base}} ((),\ \sigma \text{ with } h := \sigma.h[\ell \leftarrow \bot]\,, \epsilon)} \qquad \frac{\sigma.h(\ell) = v}{(!\,\ell, \sigma) \to_{\mathbf{base}} (v, \sigma, \epsilon)}$$

$$\frac{\sigma.h(\ell) = v}{(\ell \leftarrow w, \sigma) \to_{\mathbf{base}} ((),\ \sigma \text{ with } h := \sigma.h[\ell \leftarrow w]\,, \epsilon)}$$

$$\frac{\sigma.h(\ell) = v \qquad v \cong w_1}{(\mathbf{CmpXchg}(\ell, w_1, w_2), \sigma) \to_{\mathbf{base}} ((v, \mathbf{true}),\ \sigma \text{ with } h := \sigma.h[\ell \leftarrow w_2]\,, \epsilon)}$$

$$\frac{\sigma.h(\ell) = v \qquad v \ncong w_1}{(\mathbf{CmpXchg}(\ell, w_1, w_2), \sigma) \to_{\mathbf{base}} ((v, \mathbf{false}), \sigma, \epsilon)}$$

$$\frac{\sigma.h(\ell) = v}{(\mathbf{Xchg}(\ell, w), \sigma) \to_{\mathbf{base}} (v,\ \sigma \text{ with } h := \sigma.h[\ell \leftarrow w]\,, \epsilon)}$$

$$\frac{\sigma.h(\ell) = z_1}{(\mathbf{FAA}(\ell, z_2), \sigma) \to_{\mathbf{base}} (z_1,\ \sigma \text{ with } h := \sigma.h[\ell \leftarrow z_1 + z_2]\,, \epsilon)}$$

$$(\mathbf{fork}(e), \sigma) \to_{\mathbf{base}} ((), \sigma, [e]) \qquad \frac{p \notin \sigma.pid}{(\mathbf{newproph}, \sigma) \to_{\mathbf{base}} (p,\ \sigma \text{ with } pid := \{p\} \cup \sigma.pid, \epsilon)}$$

$$\frac{(e, \sigma) \xrightarrow{\vec{\kappa}}_{\mathbf{base}} (v, \sigma', \vec{e}\,') \qquad p \in \sigma.pid}{(\mathbf{resolve}\ e\ \mathbf{with}\ \mathbf{at}\ p\ \mathbf{to}\ w, \sigma) \xrightarrow{\vec{\kappa} +\!\!+ [(p,(v,w))]}_{\mathbf{base}} (v, \sigma', \vec{e}\,')}$$

Primitive Laws. We only include language-specific laws here. Generic wp laws are shown in Figures 2 and 3.

$$\frac{\ulcorner z > 0 \urcorner \qquad \triangleright \forall \ell.\ \left(\ast_{0 \le i < z} \ell + i \mapsto v * \mathrm{metaTok}(\ell + i, \top) \right) \mathrel{-\!\!*} \Phi(\ell)}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{AllocN}(z, v)\ \{\Phi\}} \qquad \frac{\ell \mapsto v \qquad \triangleright \Phi(())}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{free}(\ell)\ \{\Phi\}}$$

$$\frac{\mathrm{metaTok}(\ell, \mathcal{E}_1) \qquad \mathrm{metaTok}(\ell, \mathcal{E}_2)}{\ulcorner \mathcal{E}_1 \# \mathcal{E}_2 \urcorner} \qquad \frac{\ell \mapsto v \qquad \triangleright (\ell \mapsto v \mathrel{-\!\!*} \Phi(v))}{\mathrm{wp}_{\mathcal{E}}\ !\,\ell\ \{\Phi\}}$$

$$\frac{\ell \mapsto v \qquad \triangleright (\ell \mapsto w \mathrel{-\!\!*} \Phi(()))}{\mathrm{wp}_{\mathcal{E}}\ \ell \leftarrow w\ \{\Phi\}} \qquad \frac{\ell \mapsto v \qquad \ulcorner v \cong w_1 \urcorner \qquad \triangleright (\ell \mapsto w_2 \mathrel{-\!\!*} \Phi((v, \mathbf{true})))}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{CmpXchg}(\ell, w_1, w_2)\ \{\Phi\}}$$

$$\frac{\ell \mapsto v \qquad \ulcorner v \ncong w_1 \urcorner \qquad \triangleright (\ell \mapsto v \mathrel{-\!\!*} \Phi((v, \mathbf{false})))}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{CmpXchg}(\ell, w_1, w_2)\ \{\Phi\}} \qquad \frac{\ell \mapsto z_1 \qquad \triangleright (\ell \mapsto z_1 + z_2 \mathrel{-\!\!*} \Phi(z_1))}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{FAA}(\ell, z_2)\ \{\Phi\}}$$

$$\frac{\ell \mapsto v \qquad \triangleright(\ell \mapsto w \mathrel{-\!\!*} \Phi(v))}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{Xchg}(\ell, w)\ \{\Phi\}}* \qquad \frac{\triangleright \models\!\!\Rightarrow_{\mathcal{E}} (\Phi(()) * \mathrm{wp}_{\top}\ e\ \{v.\ \mathrm{True}\})}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{fork}(e)\ \{\Phi\}}* \qquad \frac{\triangleright \forall p, \vec{v}\,.\ \mathrm{proph}\ (p, \vec{v}) \mathrel{-\!\!*} \Phi(p)}{\mathrm{wp}_{\mathcal{E}}\ \mathbf{newproph}\ \{\Phi\}}*$$

$$\frac{\ulcorner \mathrm{Atomic}\ e \urcorner \qquad \mathrm{proph}\ (p, \vec{v})}{\mathrm{proph}\ (p, \vec{v}) \mathrel{-\!\!*} \mathrm{wp}_{\mathcal{E}}\ e\ \left\{r.\ \exists \vec{v}\,'.\ \mathrm{proph}\ (p, \vec{v}\,') * \forall \vec{v}\,''.\ \ulcorner \vec{v}\,' = (r, w) :: \vec{v}\,'' \urcorner \mathrel{-\!\!*} \mathrm{proph}\ (p, \vec{v}\,'') \mathrel{-\!\!*} \Phi(r)\right\}}*$$
$$\mathrm{wp}_{\mathcal{E}}\ \mathbf{resolve}\ \mathbf{with}\ e\ \mathbf{at}\ p\ \mathbf{to}\ w\ \{\Phi\}$$

E The Syntax of λ_{Rust}

Loads and stores in λ_{Rust} take an order o indicating if the operation is atomic (**sc**) or not (**na**). The order **na'** shows up during the execution of a non-atomic memory operation. As described in §6, these take two steps, with the first step acquiring the lock and changing the order to **na'**, so that the next step can then finish the operation and release the lock. This makes **na'** an administrative redex, and it is not considered part of the source syntax. The state σ implements a block-based memory model (locations are pairs of a block and an offset [46]) and equips every cell with a reader-writer lock π.

$$\begin{aligned}
e \in \mathit{Expr} &::= v \mid x \mid e_1.e_2 \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \le e_2 \mid e_1 == e_2 \mid \mathbf{alloc}(e) \mid \mathbf{free}(e_1, e_2) \quad (z \in \mathbb{Z}) \\
&\mid {}^{*o}e \mid e_1 :=_o e_2 \mid \mathbf{CAS}(e_0, e_1, e_2) \mid \mathbf{case}\ e\ \mathbf{of}\ \bar{e} \mid e(\bar{e}) \mid \mathbf{fork}(e) \\
v \in \mathit{Val} &::= \lightning \mid \ell \mid z \mid \mathbf{rec}\ f(\bar{x}) := e \qquad \ell \in \mathit{Loc} ::= (i, n) \qquad o \in \mathit{Order} ::= \mathbf{sc} \mid \mathbf{na} \mid \mathbf{na'} \\
\pi \in \mathit{LockSt} &::= \mathbf{writing} \mid \mathbf{reading}\ n \qquad (n \in \mathbb{N}) \\
\sigma \in \mathit{State} &\triangleq (\mathbb{N} \times \mathbb{N}) \xrightarrow{\mathrm{fin}} \mathit{LockSt} \times \mathit{Val} \qquad \rho \in \mathit{Conf} \triangleq \mathcal{L}(\mathit{Expr}) \times \mathit{State}
\end{aligned}$$

F The Syntax of λ_{tc}

The case study in §7 considers a language named λ_{tc}, which is a variant of ConcLang with the addition of the **tick** operation.

$$\begin{aligned}
v, w \in \mathit{Val} &::= z \mid b \mid () \mid \ell \mid \mathbf{rec}\ f(x) = e \mid (v, w) \mid \mathbf{inl}(v) \mid \mathbf{inr}(v) \qquad (z \in \mathbb{Z}, b \in \mathbb{B}, \ell \in \mathit{Loc}) \\
e \in \mathit{Expr} &::= v \mid x \mid \mathbf{rec}\ f(x) = e \mid e_1(e_2) \mid e_1 + e_2 \mid e_1 - e_2 \mid \cdots \mid \mathbf{if}\ e\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \mid (e_1, e_2) \\
&\mid \mathbf{fst}(e) \mid \mathbf{snd}(e) \mid \mathbf{inl}(e) \mid \mathbf{inr}(e) \mid (\mathbf{match}\ e\ \mathbf{with}\ \mathbf{inl}\ v \Rightarrow e_1 \mid \mathbf{inr}\ w \Rightarrow e_2\ \mathbf{end}) \\
&\mid \mathbf{ref}(e) \mid !\,e \mid e_1 \leftarrow e_2 \mid \mathbf{CAS}(e_0, e_1, e_2) \mid \mathbf{FAA}(e_1, e_2) \mid \mathbf{fork}(e) \mid \mathbf{tick}(e) \\
\sigma \in \mathit{State} &\triangleq \{h\colon \mathit{Loc} \xrightarrow{\mathrm{fin}} \mathit{Val};\ tc\colon \mathbb{N}\} \qquad \rho \in \mathit{Conf} \triangleq \mathcal{L}(\mathit{Expr}) \times \mathit{State}
\end{aligned}$$

G The Syntax of $\lambda_{\mathrm{ref}}^{\mathrm{rand}}$

The syntax of $\lambda_{\mathrm{ref}}^{\mathrm{rand}}$, the language targeted by Eris, is reproduced below from Aguirre et al. [1]. It is essentially the same as that of HeapLang, with the removal of prophecy variables and concurrency primitives, and the addition of the **rand** construct for probabilistic choice. Note that Theorem 24 only handles a subset of $\lambda_{\mathrm{ref}}^{\mathrm{rand}}$ without **AllocN**.

$$\begin{aligned}
v, w \in \mathit{Val} &::= z \mid b \mid () \mid \ell \mid \mathbf{rec}\ f(x) = e \mid (v, w) \mid \mathbf{inl}(v) \mid \mathbf{inr}(v) \qquad (z \in \mathbb{Z}, b \in \mathbb{B}, \ell \in \mathit{Loc}) \\
e \in \mathit{Expr} &::= v \mid x \mid \mathbf{rec}\ f(x) = e \mid e_1(e_2) \mid e_1 + e_2 \mid e_1 - e_2 \mid \ldots \mid \mathbf{if}\ e\ \mathbf{then}\ e_1\ \mathbf{else}\ e_2 \mid (e_1, e_2) \\
&\mid \mathbf{fst}(e) \mid \mathbf{snd}(e) \mid \mathbf{inl}(e) \mid \mathbf{inr}(e) \mid (\mathbf{match}\ e\ \mathbf{with}\ \mathbf{inl}\ v \Rightarrow e_1 \mid \mathbf{inr}\ w \Rightarrow e_2\ \mathbf{end}) \\
&\mid \mathbf{AllocN}(e_1, e_2) \mid !\,e \mid e_1 \leftarrow e_2 \mid \mathbf{rand}\ e \\
\sigma \in \mathit{State} &\triangleq (\mathit{Loc} \xrightarrow{\mathrm{fin}} \mathit{Val}) \qquad \rho \in \mathit{Conf} \triangleq \mathit{Expr} \times \mathit{State}
\end{aligned}$$

References

[1] Alejandro Aguirre, Philipp G. Haselwarter, Markus de Medeiros, Kwing Hei Li, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. 2024. Error Credits: Resourceful Reasoning about Error Bounds for Higher-Order Probabilistic Programs. *Proc. ACM Program. Lang.* 8, ICFP, Article 246 (Aug. 2024), 33 pages. doi:10.1145/3674635

[2] Clément Allain and Gabriel Scherer. 2026. Zoo: A Framework for the Verification of Concurrent OCaml 5 Programs using Separation Logic. *Proc. ACM Program. Lang.* 10, POPL (2026), 1702–1729. doi:10.1145/3776701

[3] Andrew W. Appel, Paul-André Melliès, Christopher D. Richards, and Jérôme Vouillon. 2007. A very modal model of a modern, major, general type system. In *POPL*. 109–122. doi:10.1145/1190216.1190235

[4] Robert Atkey. 2010. Amortised Resource Analysis with Separation Logic. In *ESOP (LNCS, Vol. 6012)*. Springer, 85–103. doi:10.1007/978-3-642-11957-6_6

[5] Lars Birkedal and Aleš Bizjak. 2023. Lecture Notes on Iris: Higher-Order Concurrent Separation Logic. https://iris-project.org/tutorial-material.html. https://iris-project.org/tutorial-pdfs/iris-lecture-notes.pdf Aarhus University.

[6] Lars Birkedal, Rasmus Ejlers Møgelberg, Jan Schwinghammer, and Kristian Støvring. 2012. First steps in synthetic guarded domain theory: step-indexing in the topos of trees. *Log. Methods Comput. Sci.* 8, 4 (2012). doi:10.2168/LMCS-8(4:1)2012

[7] John Boyland. 2003. Checking Interference with Fractional Permissions. In *SAS (LNCS, Vol. 2694)*. Springer, 55–72. doi:10.1007/3-540-44898-5_4

[8] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2019. Verifying concurrent, crash-safe systems with Perennial. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles, SOSP 2019, Huntsville, ON, Canada, October 27-30, 2019*. 243–258. doi:10.1145/3341301.3359632

[9] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. 2021. GoJournal: a verified, concurrent, crash-safe journaling system. In *OSDI*. 423–439. https://www.usenix.org/conference/osdi21/presentation/chajed

[10] Arthur Charguéraud. 2011. Characteristic formulae for the verification of imperative programs. *SIGPLAN Not.* 46, 9 (Sept. 2011), 418–430. doi:10.1145/2034574.2034828

[11] Krzysztof Ciesielski. 2007. On Stefan Banach and some of his results. *Banach Journal of Mathematical Analysis* 1, 1 (2007), 1–10. doi:10.15352/bjma/1240321550

[12] Stephen A. Cook. 1978. Soundness and Completeness of an Axiom System for Program Verification. *SIAM J. Comput.* 7, 1 (1978), 70–90. doi:10.1137/0207005

[13] Hoang-Hai Dang, Jacques-Henri Jourdan, Jan-Oliver Kaiser, and Derek Dreyer. 2020. RustBelt meets relaxed memory. *Proc. ACM Program. Lang.* 4, POPL (2020), 34:1–34:29. doi:10.1145/3371102

[14] Frank S. de Boer and Hans-Dieter A. Hiep. 2025. Beyond Concurrent Separation Logic: Who is Afraid of Completeness Proofs?. In *Principles of Formal Quantitative Analysis - Essays Dedicated to Christel Baier on the Occasion of Her 60th Birthday (LNCS, Vol. 15760)*, Nathalie Bertrand, Clemens Dubslaff, and Sascha Klüppelholz (Eds.). Springer, 301–319. doi:10.1007/978-3-031-97439-7_15

[15] Paulo Emílio de Vilhena and François Pottier. 2023. A Type System for Effect Handlers and Dynamic Labels. In *ESOP*. 225–252. doi:10.1007/978-3-031-30044-8_9

[16] Lennard Gäher, Michael Sammler, Ralf Jung, Robbert Krebbers, and Derek Dreyer. 2024. RefinedRust: A Type System for High-Assurance Verification of Rust Programs. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1115–1139. doi:10.1145/3656422

[17] Lennard Gäher, Michael Sammler, Simon Spies, Ralf Jung, Hoang-Hai Dang, Robbert Krebbers, Jeehoon Kang, and Derek Dreyer. 2022. Simuliris: a Separation Logic Framework for Verifying Concurrent Program Optimizations. *Proc. ACM Program. Lang.* 6, POPL (2022), 1–31. doi:10.1145/3498689

[18] Aïna Linn Georges, Benjamin Peters, Laila Elbeheiry, Leo White, Stephen Dolan, Richard A. Eisenberg, Chris Casinghino, François Pottier, and Derek Dreyer. 2025. Data Race Freedom à la Mode. *Proc. ACM Program. Lang.* 9, POPL (2025), 656–686. doi:10.1145/3704859

[19] Paolo G. Giarrusso, Léo Stefanesco, Amin Timany, Lars Birkedal, and Robbert Krebbers. 2020. Scala step-by-step: soundness for DOT with step-indexed logical relations in Iris. *Proc. ACM Program. Lang.* 4, ICFP (2020), 114:1–114:29. doi:10.1145/3408996

[20] Simon Oddershede Gregersen, Chaitanya Agarwal, and Joseph Tassarotti. 2025. Logical Relations for Formally Verified Authenticated Data Structures. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS 2025, Taipei, Taiwan, October 13-17, 2025*. 1394–1408. doi:10.1145/3719027.3744801

[21] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2024. Almost-Sure Termination by Guarded Refinement. *Proc. ACM Program. Lang.* 8, ICFP (2024), 203–233. doi:10.1145/3674632

[22] Simon Oddershede Gregersen, Alejandro Aguirre, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2024. Asynchronous Probabilistic Couplings in Higher-Order Separation Logic. *Proc. ACM Program. Lang.* 8, POPL (2024), 753–784. doi:10.1145/3632868

[23] Simon Oddershede Gregersen, Johan Bay, Amin Timany, and Lars Birkedal. 2021. Mechanized logical relations for termination-insensitive noninterference. *Proc. ACM Program. Lang.* 5, POPL (2021), 1–29. doi:10.1145/3434291

[24] Armaël Guéneau, Johannes Hostert, Simon Spies, Michael Sammler, Lars Birkedal, and Derek Dreyer. 2023. Melocoton: A Program Logic for Verified Interoperability Between OCaml and C. *Proc. ACM Program. Lang.* 7, OOPSLA2 (2023), 716–744. doi:10.1145/3622823

[25] Philipp G. Haselwarter, Kwing Hei Li, Alejandro Aguirre, Simon Oddershede Gregersen, Joseph Tassarotti, and Lars Birkedal. 2025. Approximate Relational Reasoning for Higher-Order Probabilistic Programs. *Proc. ACM Program. Lang.* 9, POPL (2025), 1196–1226. doi:10.1145/3704877

[26] Philipp G. Haselwarter, Kwing Hei Li, Markus de Medeiros, Simon Oddershede Gregersen, Alejandro Aguirre, Joseph Tassarotti, and Lars Birkedal. 2024. Tachis: Higher-Order Separation Logic with Credits for Expected Costs. *Proc. ACM Program. Lang.* 8, OOPSLA2, Article 313 (Oct. 2024), 30 pages. doi:10.1145/3689753

[27] Maximilian P. L. Haslbeck and Tobias Nipkow. 2018. Hoare Logics for Time Bounds - A Study in Meta Theory. In *TACAS (LNCS, Vol. 10805)*, Dirk Beyer and Marieke Huisman (Eds.). Springer, 155–171. doi:10.1007/978-3-319-89960-2_9

[28] Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. 2020. Actris: session-type based reasoning in separation logic. *Proc. ACM Program. Lang.* 4, POPL (2020), 6:1–6:30. doi:10.1145/3371074

[29] C. A. R. Hoare. 1974. Monitors: An Operating System Structuring Concept. *Commun. ACM* 17, 10 (1974), 549–557. doi:10.1145/355620.361161

[30] Kohei Honda, Nobuko Yoshida, and Martin Berger. 2005. An Observationally Complete Program Logic for Imperative Higher-Order Functions. In *20th IEEE Symposium on Logic in Computer Science (LICS 2005), 26-29 June 2005, Chicago, IL, USA, Proceedings*. IEEE Computer Society, 270–279. doi:10.1109/LICS.2005.5

[31] Johannes Hostert, Zichen Zhang, Puming Liu, Simon Oddershede Gregersen, Ralf Jung, and Joseph Tassarotti. 2026. Completeness of Iris-Based Program Logics (Artifact). doi:10.5281/zenodo.20625901

[32] Iris developers and contributors. 2026. The Iris 4.5 Reference. https://plv.mpi-sws.org/iris/appendix-4.5.pdf

[33] Samin S. Ishtiaq and Peter W. O'Hearn. 2001. BI as an Assertion Language for Mutable Data Structures. In *POPL*. 14–26. doi:10.1145/360204.375719

[34] Cliff B. Jones. 1983. Tentative Steps Toward a Development Method for Interfering Programs. *ACM Trans. Program. Lang. Syst.* 5, 4 (1983), 596–619. doi:10.1145/69575.69577

[35] Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers, and Derek Dreyer. 2017. RustBelt: Securing the Foundations of the Rust Programming Language. *Proc. ACM Program. Lang.* 2, POPL (2017), 1–34. doi:10.1145/3158154

[36] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Ales Bizjak, Lars Birkedal, and Derek Dreyer. 2018. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *J. Funct. Program.* 28 (2018), e20. doi:10.1017/S0956796818000151

[37] Ralf Jung, Rodolphe Lepigre, Gaurav Parthasarathy, Marianna Rapoport, Amin Timany, Derek Dreyer, and Bart Jacobs. 2019. The future is ours: prophecy variables in separation logic. *Proc. ACM Program. Lang.* 4, POPL, Article 45 (dec 2019), 32 pages. doi:10.1145/3371113

[38] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. 2015. Iris: Monoids and Invariants as an Orthogonal Basis for Concurrent Reasoning. In *POPL*. ACM, 637–650. doi:10.1145/2676726.2676980

[39] Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. 2017. Strong Logic for Weak Memory: Reasoning About Release-Acquire Consistency in Iris. In *31st European Conference on Object-Oriented Programming, ECOOP 2017, Barcelona, Spain, June 19-23, 2017*. 17:1–17:29. doi:10.4230/LIPICS.ECOOP.2017.17

[40] Robbert Krebbers, Jacques-Henri Jourdan, Ralf Jung, Joseph Tassarotti, Jan-Oliver Kaiser, Amin Timany, Arthur Charguéraud, and Derek Dreyer. 2018. MoSeL: A general, extensible modal framework for interactive proofs in separation logic. *PACMPL* 2, ICFP (2018), 77:1–77:30. doi:10.1145/3236772

[41] Robbert Krebbers, Ralf Jung, Ales Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. 2017. The Essence of Higher-Order Concurrent Separation Logic. In *ESOP (LNCS, Vol. 10201)*. Springer, 696–723. doi:10.1007/978-3-662-54434-1_26

[42] Robbert Krebbers, Amin Timany, and Lars Birkedal. 2017. Interactive proofs in higher-order concurrent separation logic. In *POPL*. 205–217. doi:10.1145/3009837.3009855

[43] Robbert Krebbers, Luko van der Maas, and Enrico Tassi. 2025. Inductive Predicates via Least Fixpoints in Higher-Order Separation Logic. In *ITP (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 352)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:21. doi:10.4230/LIPIcs.ITP.2025.27

[44] Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. 2020. Aneris: A Mechanised Logic for Modular Reasoning about Distributed Systems. In *ESOP*. 336–365. doi:10.1007/978-3-030-44914-8_13

[45] Peter John Landin. 1964. The Mechanical Evaluation of Expressions. *Comput. J.* (1964). doi:10.1093/COMJNL/6.4.308

[46] Xavier Leroy, Andrew Appel, Sandrine Blazy, and Gordon Stewart. 2012. *The CompCert memory model, version 2.* Technical Report RR-7987. Inria. https://hal.inria.fr/hal-00703441

[47] Kwing Hei Li, Alejandro Aguirre, Simon Oddershede Gregersen, Philipp G. Haselwarter, Joseph Tassarotti, and Lars Birkedal. 2025. Modular Reasoning about Error Bounds for Concurrent Probabilistic Programs. *Proc. ACM Program. Lang.* 9, ICFP, Article 245 (Aug. 2025), 30 pages. doi:10.1145/3747514

[48] Martin Hugo Löb. 1955. Solution of a problem of Leon Henkin. *Journal of Symbolic Logic* 20, 2 (1955), 115–118. doi:10.2307/2266895

[49] Rupak Majumdar and V. R. Sathiyanarayana. 2025. Sound and Complete Proof Rules for Probabilistic Termination. *Proc. ACM Program. Lang.* 9, POPL (2025), 1871–1902. doi:10.1145/3704899

[50] William Mansky and Ke Du. 2024. An Iris Instance for Verifying CompCert C Programs. *Proc. ACM Program. Lang.* 8, POPL (2024), 148–174. doi:10.1145/3632848

[51] Yusuke Matsushita, Xavier Denis, Jacques-Henri Jourdan, and Derek Dreyer. 2022. RustHornBelt: a semantic foundation for functional verification of Rust programs with unsafe code. In *PLDI*. 841–856. doi:10.1145/3519939.3523704

[52] Annabelle McIver, Carroll Morgan, Benjamin Lucien Kaminski, and Joost-Pieter Katoen. 2018. A new proof rule for almost-sure termination. *Proc. ACM Program. Lang.* 2, POPL (2018), 33:1–33:28. doi:10.1145/3158121

[53] Glen Mével and Jacques-Henri Jourdan. 2021. Formal verification of a concurrent bounded queue in a weak memory model. *Proc. ACM Program. Lang.* 5, ICFP (2021), 1–29. doi:10.1145/3473571

[54] Glen Mével, Jacques-Henri Jourdan, and François Pottier. 2019. Time Credits and Time Receipts in Iris. In *Programming Languages and Systems*, Luís Caires (Ed.). Springer International Publishing, Cham, 3–29. doi:10.1007/978-3-030-17184-1_1

[55] Alexandre Moine, Stephanie Balzer, Alex Xu, and Sam Westrick. 2026. TypeDis: A Type System for Disentanglement. *Proc. ACM Program. Lang.* 10, POPL (2026), 354–383. doi:10.1145/3776655

[56] Alexandre Moine, Arthur Charguéraud, and François Pottier. 2023. A High-Level Separation Logic for Heap Space under Garbage Collection. *Proc. ACM Program. Lang.* 7, POPL (2023), 718–747. doi:10.1145/3571218

[57] Hiroshi Nakano. 2000. A Modality for Recursion. In *15th Annual IEEE Symposium on Logic in Computer Science, Santa Barbara, California, USA, June 26-29, 2000*. 255–266. doi:10.1109/LICS.2000.855774

[58] Susan S. Owicki. 1975. *Axiomatic Proof Techniques for Parallel Programs.* Ph. D. Dissertation. Cornell University.

[59] Susan S. Owicki and David Gries. 1976. An Axiomatic Proof Technique for Parallel Programs I. *Acta Informatica* 6 (1976), 319–340. doi:10.1007/BF00268134

[60] François Pottier, Armaël Guéneau, Jacques-Henri Jourdan, and Glen Mével. 2024. Thunks and Debits in Separation Logic with Time Credits. *Proc. ACM Program. Lang.* 8, POPL (2024), 1482–1508. doi:10.1145/3632892

[61] Michael Sammler, Rodolphe Lepigre, Robbert Krebbers, Kayvan Memarian, Derek Dreyer, and Deepak Garg. 2021. RefinedC: Automating the Foundational Verification of C Code with Refined Ownership Types. In *PLDI*. 158–174. doi:10.1145/3453483.3454036

[62] Remy Seassau, Irene Yoon, Jean-Marie Madiot, and François Pottier. 2025. Formal Semantics and Program Logics for a Fragment of OCaml. *Proc. ACM Program. Lang.* 9, ICFP (2025), 128–159. doi:10.1145/3747509

[63] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. 2023. Grove: a Separation-Logic Library for Verifying Distributed Systems. In *SOSP*. 113–129. doi:10.1145/3600006.3613172

[64] Colin Stirling. 1988. A Generalization of Owicki-Gries's Hoare Logic for a Concurrent while Language. *Theor. Comput. Sci.* 58 (1988), 347–359. doi:10.1016/0304-3975(88)90033-3

[65] Kasper Svendsen and Lars Birkedal. 2014. Impredicative Concurrent Abstract Predicates. In *ESOP (LNCS, Vol. 8410)*. Springer, 149–168. doi:10.1007/978-3-642-54833-8_9

[66] The Rocq Team. 2026. The Rocq Prover. https://rocq-prover.org/.

[67] Amin Timany, Simon Oddershede Gregersen, Léo Stefanesco, Jonas Kastberg Hinrichsen, Léon Gondelman, Abel Nieto, and Lars Birkedal. 2024. Trillium: Higher-Order Concurrent and Distributed Separation Logic for Intensional Refinement. *Proc. ACM Program. Lang.* 8, POPL (2024), 241–272. doi:10.1145/3632851

[68] Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. 2024. A Logical Approach to Type Soundness. *J. ACM* 71, 6 (2024), 40:1–40:75. doi:10.1145/3676954

[69] Aaron Turon, Derek Dreyer, and Lars Birkedal. 2013. Unifying refinement and hoare-style reasoning in a logic for higher-order concurrency. In *ICFP*. ACM, 377–390. doi:10.1145/2500365.2500600

[70] Orpheas van Rooij and Robbert Krebbers. 2025. Affect: An Affine Type and Effect System. *Proc. ACM Program. Lang.* 9, POPL (2025), 126–154. doi:10.1145/3704841

[71] Qiwen Xu, Willem P. de Roever, and Jifeng He. 1997. The Rely-Guarantee Method for Verifying Shared Variable Concurrent Programs. *Formal Aspects Comput.* 9, 2 (1997), 149–174. doi:10.1007/BF01211617

[72] Hongseok Yang. 2001. *Local reasoning for stateful programs.* Ph. D. Dissertation. USA. Advisor(s) Reddy, Uday S. AAI3023240.

[73] Hongseok Yang and Peter W. O'Hearn. 2002. A Semantic Basis for Local Reasoning. In *Proceedings of the 5th International Conference on Foundations of Software Science and Computation Structures (FoSSaCS '02)*. Springer-Verlag, Berlin, Heidelberg, 402–416. doi:10.1007/3-540-45931-6_28

Received 2026-02-19; accepted 2026-05-13